

Vehicle Damage Recognition System

John Wensink

CSC510 Foundations of Artificial Intelligence

Colorado State University – Global Campus

Dr. Bingdong Li

March, 7, 2025

Introduction to The Vehicle Damage Recognition System

The Vehicle Damage Recognition System (VDRS) is a Convolutional Neural Network (CNN) designed to analyze images of vehicles and classify damage types, such as no damage, scratches, dents, shattered glass, total loss, and more (Padmanabhan, R., Padmanabhan, D. R., & Thanigaivelu, 2023.) This AI-driven solution functions as a Decision Support System (DSS) for an insurance company (Fairlie, 2025), assisting in the classification of user-submitted photos. By automating damage identification, VDRS helps streamline the early stages of the claims process through symbolic planning, delivering preliminary damage assessments to adjusters after the first notice of loss. Symbolic planning enables the system to represent vehicle damages, repair categories, and decision-making rules in a structured format, allowing for logical reasoning about what the most appropriate next step should be, enhancing workflow efficiency and minimizing initial claim processing time. VDRS uses deep learning techniques, utilizing a pre-trained ResNet152 (GeeksforGeeks, 2025) model fine-tuned to recognize vehicle damage patterns based on a Kaggle dataset (Manssor, n.d.) of 1,512 labeled images. The training images correspond to a CSV file, which maps each image file name to its corresponding damage class. The dataset is split into training and validation sets using SciKit-Learn's *train_test_split* library (SciKit-Learn Developers, n.d.), and uses stratified sampling to maintain a proportional distribution of damage classes in both subsets. This ensures that the model is trained on a balanced representation of all damage categories, preventing class imbalance from skewing predictions (Analytics Vidhya, 2024). The training set is used to optimize the model parameters, and the validation set allows for performance evaluation and hyperparameter tuning. To improve model generalization, the dataset goes through preprocessing and augmentation (TorchVision

Developers, n.d.). Preprocessing steps include resizing each image to 224x224 pixels, color normalization, and conversion to tensors to match the input requirements of ResNet-152 (GeeksforGeeks, 2025). Data augmentation techniques such as random cropping, horizontal flipping, affine transformations, and brightness adjustments are applied to simulate real-world variation in vehicle damage, reducing the likelihood of overfitting (Google Developers, n.d.)

The ResNet-152-based model architecture retains its convolutional layers for feature extraction but replaces the default fully connected (FC) layer with a custom classification head. The new FC layer consists of a dropout regularization layer followed by a linear classifier, which maps extracted features to predefined categories. The dropout layer helps prevent overfitting by reducing reliance on specific neurons during training, improving generalization with unseen user-provided images. The linear classifier enables precise mapping of extracted features to damage categories, ensuring the model effectively differentiates between various types of vehicle damage. By adapting the FC layer, the model is fine-tuned for domain-specific classification while providing the advanced feature extraction capabilities of ResNet-152.

The model trains for 50 epochs, with performance evaluated (Sling Academy, 2024) at each epoch based on training loss, validation loss, and validation accuracy. This project uses a progressive layer unfreezing strategy to fine-tune the pre-trained ResNet-152 model where, initially, all layers of the model are frozen to preserve the general feature extraction capabilities learned from ImageNet. As training progresses, deeper layers of the model are incrementally unfrozen to capture higher-level abstract features. Similarly, a learning rate scheduler dynamically adjusts training rates based on model performance to improve convergence. A very small learning rate (0.00003) is used with the Adam optimizer (Kingma & Ba, 2017), which

helps achieve smooth and gradual weight updates and reduces the risk of overshooting minima during optimization. The ***ReduceLROnPlateau*** (PyTorch, 2024b) scheduler is implemented to adjust the learning rate based on validation performance. If validation accuracy fails to improve over multiple epochs, the scheduler reduces the learning rate by a factor of 0.5, allowing the model to fine-tune its weights more precisely in later stages of training. To ensure optimal performance, the system saves the model's weights whenever an improvement is detected in validation accuracy and loss, allowing it to retain the best-performing version throughout the training process. The training takes approximately 12 minutes running on an NVIDIA 3080 GPU.

Once the training is complete, a separate Python script utilizes the saved model weights to classify new images provided by users. This script loads the trained ResNet-152 model, applies the same preprocessing pipeline, and runs inference on the uploaded image. The model processes the image and outputs a predicted damage classification, mapping it to one of the predefined categories. To ensure consistency between training and inference, the classification script follows the same preprocessing steps, normalizing the user-provided image into a 224x224 pixel input, and converting that into a tensor. VDRS is intended to run PyTorch (2022) on a CUDA-enabled GPU if available but also supports CPU-based execution for broader accessibility. The model is set to evaluation mode to ensure deterministic predictions, and inference is performed without computing gradients to optimize efficiency.

During the development of this project, I utilized several large language models to generate architectural foundations, which I then modified, optimized, and tailored to meet the specific requirements of my project. I followed the waterfall approach to project management,

beginning with the development of very simple CNNs and progressively testing and sometimes incorporating additional features. This structured, step-by-step methodology allowed me to refine the model at each stage, ultimately resulting in a more advanced and relatively effective system capable of accurately classifying user-provided images of vehicle damage.

In the following sections, I will explain in detail how this model works, the development history of this project, and the challenges encountered during implementation.

```

1  import torch
2  import torch.nn as nn
3  import torch.optim as optim
4  import torchvision.models as models
5  from torchvision import transforms
6  from torch.utils.data import Dataset, DataLoader
7  from PIL import Image
8  import pandas as pd
9  import os
10 import numpy as np
11 from sklearn.model_selection import train_test_split
12 from sklearn.utils.class_weight import compute_class_weight
13
14 # Paths
15 CSV_PATH = r"C:\Users\jwens\Desktop\CSUGlobal\CSC510\CSC510 Portfolio Project\CSC510 Portfolio Project\
16 IMAGE_DIR = r"C:\Users\jwens\Desktop\CSUGlobal\CSC510\CSC510 Portfolio Project\CSC510 Portfolio Project\
17
18 # Set device
19 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
20 print(f"Using device: {device}")
21
22 # Load CSV
23 df = pd.read_csv(CSV_PATH)
24 df["image"] = df["image"].str.replace("image/", "", regex=False)
25
26 # Encode labels into numeric values
27 class_mapping = {label: idx for idx, label in enumerate(df["classes"].unique())}
28 df["label"] = df["classes"].map(class_mapping)
29
30 # Print class mapping and distribution to check for imbalance
31 print("Class mapping:")
32 for class_name, idx in class_mapping.items():
33     print(f"{idx}: {class_name}")
34
35 class_counts = df["classes"].value_counts()
36 print("\nClass distribution:")
37 print(class_counts)
38
39 # Split dataset
40 train_df, val_df = train_test_split(df, test_size=0.2, stratify=df["label"], random_state=42)
41
42 # **Data Augmentation** (unchanged from your original code)
43 transform = transforms.Compose([
44     transforms.Resize((224, 224)),
45     transforms.RandomResizedCrop(size=224, scale=(0.8, 1.0)),
46     transforms.RandomHorizontalFlip(p=0.5),
47     transforms.RandomAffine(degrees=15, translate=(0.1, 0.1), scale=(0.9, 1.1), shear=10),
48     transforms.ColorJitter(brightness=0.3, contrast=0.3),
49     transforms.ToTensor(),
50     transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
51 ])
52
53 # **Dataset Class**
54 class VehicleDamageDataset(Dataset):
55

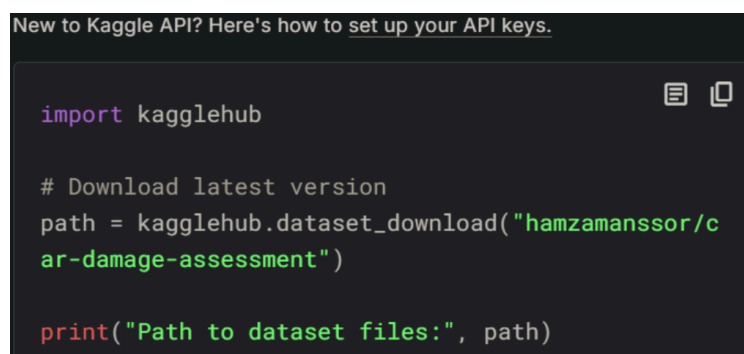
```

System Design and Implementation

In this section, I will outline the key components and workflow of the system, detailing the dataset used for training, the model architecture, the various attempts at optimizing the system, as well as final steps to prepare the system for deployment.

Training Dataset

The VDRS model was trained on the Car Damage Assessment dataset from Kaggle (Manssor, n.d.), which contains 1512 images of vehicle damage and a CSV file with damage labels corresponding to the image file names. To get this data, I needed to set up a Kaggle API key in the form of a JSON file from Kaggle's website (Kaggle, n.d.) This was to be the first file in my project, so at this point, I started a new PyCharm project with a virtual environment (venv) in order to keep my dependencies isolated for this undertaking. In order to import the dataset, I used PyCharm's terminal to run *pip install kaggle* and created a *config* subfolder in my project's main folder where I placed my API key. To download the dataset, I used a simple Python script with the variables mapped to the destination folder and dataset I was seeking.



```
New to Kaggle API? Here's how to set up your API keys.

import kagglehub

# Download latest version
path = kagglehub.dataset_download("hamzamanssor/c
ar-damage-assessment")

print("Path to dataset files:", path)
```

(Kaggle, 2025)

I ran into trouble because the Kaggle API is preconfigured to search for the JSON file in a default directory (Kaggle, n.d.) Grimoire (2025) helped me figure out what the issue was, and then I was able to view the data.

I didn't realize it at the time, but there were about 500 images that were classified as 'unknown.' This became a problem later when I was trying to get my model's accuracy and loss to an acceptable level. To remedy this problem, I made a Python script that copied all the images with an 'unknown' label to a separate folder. The script was programmed to open the image, prompt me for input on which category the image should be (1 = no damage, 2 = total loss, etc), update that image's CSV row, and finally delete the image from the temporary folder. I found most of the images should have belonged to either no damage or total loss, and I went through each image 'manually' adding the correct label. This led to a substantial improvement in model accuracy because the features of an image with 'no damage' were quite different than the features of an image with a 'total loss.' Final accuracy scores for running the model with only the unaugmented training data plateaued at around 50%. I noticed that the damage categorization classes were unevenly distributed, causing class imbalance issues. To remedy this, I added functionality for a weighted loss function to give more weight to underrepresented classes. This improved accuracy a bit, up to around 60%.

Overall, I would describe the training data as poor quality. Although 1,500 images might sound like a lot of data, realistically, I would have liked to have had many more images and a broader damage category set to create a model suitable for real-world performance. With that being said, there are some techniques that I used to artificially improve the quality of my training/validation dataset. Data augmentation techniques like cropping, rotations (Stack

Exchange Community, 2020), brightness/contrast adjustments, affine transformations (TorchVision Developers, n.d.), and color jittering actually added quite a bit of variety to the dataset and substantially improved the model's generalization. I was pleasantly surprised at just how much improvement was possible, expecting the 'garbage in, garbage out' mantra to hold true. However, I found that applying these transforms from the *torchvision* library was able to counteract a lot of the limitations of the dataset and make it more representative of real-world scenarios.

While data augmentation was certainly shown to be a powerful tool, it was ResNet that was doing a lot of the heavy lifting. ResNet's ability to learn hierarchical features at multiple levels makes it a particularly well-suited tool for image classification tasks like this one. The lower layers focus on more basic features, such as edges and textures, while the deeper layers are able to capture more abstract representations of vehicle parts and the nature of damages. I started out using ResNet18, which was trained on ImageNet's 1.3 million images (Gravikov, 2022). Since the pre-trained layers wouldn't need large changes, I decreased the learning rate from 0.001 to 0.0001 in order to prevent the model from 'unlearning' the pre-trained features, only fine-tuning the last layer to learn my specific dataset. I immediately noticed an improvement in accuracy, jumping suddenly up to 85%. This can be attributed to ResNet being a pre-trained model that already does pretty well with general image features like edges, textures, and shapes. With the goal of getting to 90% validation accuracy, I explored more powerful versions of ResNet, experimenting with ResNet versions 34 (21M features), 50 (25M features), 101 (44M features), and 152 (60M features) and found that accuracy improved when using the more powerful frameworks. The tradeoff here is time, with more features being analyzed taking longer

to train. With ResNet152 backing the augmented training dataset, accuracy at classifying the validation set reached as high as 89.66%.

The Kaggle dataset was split into training and validation data using SciKit-Learn's *train_test_split* library. I initially set the split to keep 80% of the dataset as training data and 20% as validation data. I noticed as the model was going through its epochs that the training accuracy percentage continued to improve during the later epochs, whereas the validation accuracy percentage seemed to plateau at ~89%. I wanted to see if changes in the split percentage would improve validation accuracy, so I tried a 70/30 split as well as a 90/10 split. I found that 70/30 performed worse at about 84%, and 90/10 also performed worse at about 87%. I tried an 85/15 split, which also returned 87%. I eventually set the split back to the original 80/20, as this seems to be optimal.

At this point, I was happy with the 89.66% accurate validation rate, and I believe this is likely to be the limit of how well the model can perform given the constraints of the dataset quality. Additional fine-tuning techniques, such as progressive unfreezing, dropout adjustments, and dynamic learning rates were explored, but none produced a significant breakthrough beyond this threshold. Given the limitation in dataset size, class balance, and inherent variability in vehicle damages, achieving a higher accuracy would likely require a larger, more diverse dataset rather than refinements to the training architecture. There are some possibilities that went unexplored, such as EfficientNet (HuggingFace, n.d.) or vision transformers. For the scope of this project, the model performs reasonably well within the provided categories.

VDRS Libraries and Tools

VDRS relies on a combination of deep learning frameworks, data processing libraries, and optimization tools to facilitate model training, evaluation, and inference. The primary deep learning framework used in this project is PyTorch (2022), which provides dynamic computation graphs, GPU acceleration, and a modular approach to building and fine-tuning neural networks. I had wanted to use TensorFlow (Sharma, 2023) for this project, as it is the industry standard framework in my field of work. I tried different versions of CUDA Toolkit, cuDNN, historic driver update versions, Windows Subsystem for Linux (WSL), virtualized Linux OS', and even a clean Windows installation (sublimE__9, 2024.) No matter what I tried, I was unable to get TensorFlow to recognize and run on my GPU, and I ultimately had to abandon the effort. The troubleshooting process proved to be a drain on valuable time that could have been better spent on learning and refining the model itself. PyTorch (2022), on the other hand, recognized my hardware right away, as its streamlined setup and flexibility enabled me to focus on developing and optimizing the VDRS rather than battling compatibility issues.

PyTorch serves as the foundational framework for this deep learning network, providing essential modules for model development, training, and evaluation. The flexibility and modularity of PyTorch enable seamless integration of CNNs, making it an ideal choice for fine-tuning ResNet152. The project begins by importing torch, which grants access to PyTorch's core functionalities, including tensor operations, model construction, and GPU acceleration. The *torch.nn* module is utilized for building and modifying the neural network architectures. In this

project, it is used to customize the fully connected (FC) layer followed by a linear layer that maps extracted features to predefined damage categories. The dropout layer reduces overfitting by preventing the model from becoming too reliant on specific neurons during training, thereby improving generalization on the unseen data. For model optimization, the *torch.optim* module is used to implement the Adam optimizer. Adam combines the benefits of momentum-based gradient descent and adaptive learning rates, ensuring smooth convergence during training (Goyal, 2021.) The learning rate is initially set to 0.00003, with a *ReduceLROnPlateau* scheduler that dynamically adjusts it based on validation performance. If validation accuracy plateaus, the scheduler reduces the learning rate by a factor of 0.5, refining the model's ability to fine-tune its weights for better performance. PyTorch (2024c) also provides tools for dataset handling and efficient batch processing through *from torch.utils.data import Dataset, DataLoader*. The *Dataset* class is extended to create a custom dataset loader that reads images from the training set, applies preprocessing transformations (Sling Academy, 2024) using TorchVision, and assigns corresponding labels. The *DataLoader* class enables efficient mini-batch training, reducing memory constraints and improving computational efficiency (PyTorch, 2021.) During training, batch sizes of 32 images are processed simultaneously, leveraging GPU acceleration (*torch.cuda*) for speed optimization. Overall, PyTorch plays an important role in this project by enabling deep learning model customization, optimizing training performance, and efficiently managing data. The combination of *torch.nn* for dataset management ensures that VDRS can process and classify vehicle damage images with high accuracy and efficiency.

In conjunction with PyTorch, Torchvision is used to import the ResNet152 model, as well as perform image preprocessing tasks like resizing, normalization, and data augmentation through *torchvision.transforms* (TorchVision Developers, n.d.) Beyond model integration, *transforms* is a key component of the image preprocessing pipeline. Before images are fed into the network, they undergo several transformations to ensure consistency and enhance model generalization. These include resizing images to the standard input dimensions to match ResNet's expected format. The images are then normalized using ImageNet's *mean=[0.485, 0.456, 0.406]* and *std=[0.229, 0.224, 0.225]* (laridzhang, 2019) to align pixel intensity distributions with those seen during the model's pretraining phase. This ensures that the network can effectively interpret images without being skewed by variations in brightness and contrast. To improve the model's ability to generalize across real-world vehicle damage scenarios, Torchvision provides data augmentation techniques to artificially increase dataset diversity, as described above. Random cropping and horizontal flipping introduce variations in image framing and orientation, mimicking different perspectives of vehicle damage as would be expected from user-submitted photos. Affine transformations, including rotation and slight translations, allow the model to learn damage patterns independent of exact positioning (Stack Exchange Community, n.d.) Color jittering is applied to simulate different lighting conditions, ensuring consistency with differing environmental variations. By taking advantage of these augmentation techniques, VDRS reduces the likelihood of overfitting, allowing the model to perform well on new, unseen data. Overall, Torchvision is an indispensable part of this project, making possible model integration, preprocessing consistency, and dataset augmentation. Its tools ensure that the VDRS model receives well-structured input data and learns from a varied and realistic dataset.

The Python Imaging Library (PIL), specifically its modern fork Pillow, is another important tool in VDRS for handling and preprocessing images before they are fed into the deep learning model (Clark & Contributors, n.d.) The library is imported using *from PIL import Image*, allowing the system to load, manipulate, and convert image files into a format suitable for PyTorch-based inference. During training and inference, PIL is used to open images from the dataset directory and convert them into RGB format to ensure consistency across different file types (Clark & Contributors, n.d.). Many image datasets contain images in varying formats, such as JPEG, PMG, or BMP, which may have different color modes (i.e. grayscale, CMYK, etc.) Converting all images to RGB standardizes inputs, preventing errors caused by incompatible formats. Once an image is loaded using *Image.open(image_path).convert("RGB")*, it is then passed through the TorchVision preprocessing pipeline. In the inference script, PIL is used to load user-submitted images before they are classified by the ResNet152 model. The same preprocessing steps are applied to ensure that the image is formatted identically to those seen during training. This consistency is crucial in preventing distribution shifts (Bhattacharjee et al., 2024) which could lead to incorrect predictions. Overall, PIL (Pillow) is a very important library in VDRS, enabling seamless image loading, format conversion, and preprocessing for both training and inference. By integrating PIL with PyTorch and TorchVision, the system ensures that images are correctly formatted and standardized, allowing the model to accurately classify vehicle damage types.

The Pandas library (*import pandas as pd*) plays an important role in VDRS by providing powerful tools for data handling, preprocessing, and manipulation. In this project, Pandas is primarily used to load, organize, and process the dataset's metadata, which maps image

filenames to their corresponding damage classification labels. The dataset consists of 1,512 images stored in a CSV file. Using Pandas, this data is imported into a DataFrame *`pd.read_csv(CSV_PATH)`*, allowing for efficient manipulation and analysis. Each row in the CSV contains an image filename and its corresponding damage category. Since the dataset is stored in structured tabular format, Pandas makes it easy to filter, modify, and clean the data before feeding it into the deep learning model. One of the key preprocessing steps involves encoding categorical labels into numerical values. Since ML models operate on numerical data, the damage categories must be converted into integers. This is achieved using a label mapping directory where each unique class is assigned a corresponding integer value. By mapping categorical labels to integers, Pandas ensures that the dataset is formatted correctly for training a neural network. Additionally, Pandas is used to handle mislabeled or missing data. During the dataset exploration phase, when I noticed that around 500 images were labeled as ‘unknown,’ I used Pandas to create a custom script that allowed for easy manual relabeling. Overall, Pandas is important in VDRS because it provides an efficient data loading, transformation, and organization system. Its capabilities enable the system to seamlessly manage large datasets, preprocess labels, and ensure that the deep learning model is trained on clean, structured, and well-balanced data.

The OS module (*`import os`*) is a fundamental component of VDRS as it provides tools for file handling, directory navigation, and dynamic path management. Since the project involves processing thousands of images stored in a directory, OS is used extensively to interact with the filesystem, ensuring that images are correctly located, accessed, and processed. One of the primary uses for OS in this project is to construct dynamic file paths. Since the dataset consists

of image filenames stored in a CSV file, these names must be combined with the dataset's image directory path to access each file. This is done using *os.path.join()*, which ensures compatibility across different operating systems. Additionally, OS is used to verify the existence of files and directories before attempting to load them. This helps to prevent runtime errors caused by missing images or incorrect file paths. During dataset preprocessing, OS is also used to filter, rename, and move the mislabeled images. The script I wrote used OS to iterate through the dataset, copy mislabeled images to a temporary folder, and prompt for manual relabeling. Once relabeled, the script updated the CSV file and deleted the temporary folder's copies of the mislabeled images. For model checkpointing, OS is used to define paths for saving and loading the model weight files, ensuring that the best-performing model is retained and can be reloaded for later use in inference without having to run the training every time. Since the deep learning model is saved whenever an improvement in validation accuracy or loss is detected, OS is used alongside *torch.save()* by the line *os.path.join()*. By structuring the checkpoint as a dictionary, not only are the model weights preserved but also the optimizer state as well as label mappings, making it easier to resume training or deploy the model for inference. The *checkpoints* dictionary is created if it does not already exist, ensuring that all saved models are stored in a dedicated location. When loading the trained model for inference, OS is used to ensure that the correct file path is retrieved before attempting to load the model weights, preventing file-not-found errors and ensuring that the inference script loads the correct model. Additionally, the use of *map_location=device* allows the model to be loaded on either a GPU or a CPU, depending on the available hardware. Overall, OS is important in model checkpointing, enabling structured storage, efficient loading, error handling, and hardware management. By integrating

OS with PyTorch, VDRS ensures that the best-trained model is always available for deployment, improving reliability and cutting down on unnecessary training time.

The NumPy library (***import numpy as np***) plays an important role in VDRS by providing efficient numerical operations, data handling, and array manipulation. Since deep learning models rely on numerical computations for optimization and inference, NumPy is used extensively to process dataset labels, compute class weights, and facilitate mathematical operations throughout the project. One of the key applications of NumPy in this project is computing class weights to handle class imbalance. Since the dataset contains unevenly distributed damage categories, certain classes (i.e., ‘no damage’) have significantly more images than others (i.e., ‘glass shatter.’) This can lead to a model that overfits dominant classes and ignores underrepresented ones. NumPy is used alongside SciKit-Learn’s ***compute_class_weight*** module to extract all unique class labels, ensuring that the class weighting process remains dynamic and adaptable to different datasets. The computed weights are then converted into a PyTorch tensor so they can be integrated into the cross-entropy loss function, allowing the model to prioritize learning from underrepresented classes. NumPy is also used for dataset handling and preprocessing, particularly when converting Pandas DataFrame columns into NumPy arrays for efficient manipulation. For example, label encoding is stored as a NumPy array to speed up operations using ***labels = np.array(df[“label”])***. Furthermore, NumPy enables efficient mathematical operations in data augmentation, tensor calculations, loss tracking, and accuracy evaluation. Since deep learning models perform millions of computations each epoch, NumPy’s optimized array operations significantly reduce computational overhead compared to Python’s native list format. Overall, NumPy is an important tool in VDRS because it enables fast

computations, class balancing, and array operations. By taking advantage of NumPy's efficiency, the system ensures optimized dataset processing, improved model training balance, and reduced computational overhead, ultimately contributing to the high performance of VDRS in training its vehicle classification model.

VDRS uses SciKit-Learn (*sklearn*) for preprocessing tasks like splitting the dataset into training and validation subsets, as well as the aforementioned class weight computation. Out of all the tools I've taken away from this course, SciKit-Learn is my favorite. The resource is an absolute wealth of powerful and easy-to-use ML functions that simplify complex processes. In VDRS, SciKit-Learn plays an important role in structuring the dataset using the *train_test_split* function. This allows the labeled dataset to split in a way that ensures an even distribution of damage categories between training and validation subsets. Without this model, class imbalance would be a serious issue that could lead to biased predictions and poor generalizations. To maintain a proportional distribution of damage categories across both subsets, the code

```
train_df, val_df = train_test_split(df, test_size=0.2, stratify=df["label"], random_state=42)
```

prevents issues where the model might overfit classes that are overrepresented in the training data while performing poorly on underrepresented classes in validation. Overall, using SciKit-Learn in VDRS ensures that the model is trained on a balanced dataset while maintaining a diverse validation set for accurate performance evaluation.

Code Methods Review

Now that we understand what libraries and dependencies need to be imported, we're ready to take a closer look at what each line of this relatively complex system does. We'll start by examining how the dataset is structured and processed, including how images are loaded, transformed, and mapped to their respective damage classifications. From there, we will walk through the architecture of the ResNet152 model, discussing how transfer learning (Chilamkurthy, 2017) is used to fine-tune the pre-trained network for the specific task of vehicle damage classification. Next, we will analyze the training loop, batch processing, optimization techniques, and loss calculations to see why they were designed the way they are to improve model accuracy while preventing overfitting. We'll also explore the model checkpointing system, which ensures that the best-performing version of the model is saved for later use. Finally, we will break down the inference script, showing how the trained model processes unseen user-submitted images and outputs a classification result in real time.

```
14 # Path Directories
15 CSV_PATH = r"C:\Users\jwens\Desktop\CSUGlobal\CSC510\CSC510 Portfolio Project\CSC510 Portfolio Project\data\data.csv"
16 IMAGE_DIR = r"C:\Users\jwens\Desktop\CSUGlobal\CSC510\CSC510 Portfolio Project\CSC510 Portfolio Project\data\image"
```

Path directories. The line for *CSV_PATH* points to the CSV file, which contains metadata about the dataset, including image filenames and their corresponding damage classification labels. The *IMAGE_DIR* variable represents the directory where all images are stored. When loading an image for training or inference, the filename from the CSV is concatenated with this directory using *os.path.join()* to generate the full file path dynamically. By structuring the project this way, VDRS ensures that all dataset files are easily accessible and

organized, and allows for flexibility, making it easy to update the dataset locations without modifying the core logic of the program. If you plan on running this yourself, you will need to make sure you remove any extra quotation marks that will appear when you select Copy as Path when right-clicking on Windows.

```
18 # Set device as GPU (faster) or CPU (slower)
19 device = torch.device("cuda" if torch.cuda.is_available() else "CPU")
20 print(f"Using device: {device}")
```

Hardware availability and selection. These lines determine whether VDRS will run on a GPU (CUDA) or a CPU based on the availability of a CUDA-enabled device. Not everybody has an NVIDIA GPU. For example, if you are using an AMD or Intel GPU or a graphics card integrated with your CPU, this check will default to running on the CPU, meaning that the model will run on the computer's processor instead of a GPU. Running a deep learning model on CUDA (Compute Unified Device Architecture) is significantly faster than running it on a CPU because GPUs are optimized for parallel computations. CUDA, specifically, is NVIDIA's parallel computing platform that allows deep learning models to take advantage of the massively parallel architecture of a GPU's specialized tensor cores. Whereas a typical CPU might have 4-16 cores designed for sequential task execution, a GPU like the RTX 3080 has 8,704 CUDA cores as well as 272 Tensor Cores, allowing for thousands of parallel operations. Tensor cores are specialized processors inside NVIDIA GPUs designed to accelerate matrix multiplication, which is heavily used in deep learning models and the cores are optimized for PyTorch and TensorFlow. This makes running VDRS on a GPU approximately 20-30 times faster than if it were run on a CPU.

When training takes 12 minutes on a GPU, it could easily take several hours on a CPU. If VDRS recognizes you have a CUDA-capable GPU, it will print its name at the top of the output, if not, VDRS will default to using your CPU and print “CPU” instead.

```
"C:\Users\jwens\Desktop\CSUGlobal\CSC510\CSC510 Portfolio Project\.venv\Scripts\python.exe"  
Using device: cuda
```

This is why I spent so much time trying to get TensorFlow to recognize my GPU. In case you are planning on running this model yourself and don’t have a CUDA-capable GPU, I have included the weights from the best model as a .pth file so that you won’t have to run the training script at all for the separate classifier script to work.

```
22     # Load CSV  
23     df = pd.read_csv(CSV_PATH)  
24     df["image"] = df["image"].str.replace("image/", "", regex=False)
```

Loading the CSV and image path. VDRS begins its data preprocessing pipeline by loading the dataset’s CSV file and modifying filenames to ensure correct file path handling. The first step is reading the CSV file with Pandas read function using *df = pd.read_csv(CSV_PATH)*. This imports the dataset into a Pandas DataFrame (*df*.) The CSV (comma-separated value) file contains all the image metadata labels, including the image number and its corresponding classification label. The *image* column in the CSV typically stores file names in a structured format, normally with a subdirectory path such as “*image/car1.jpg*.” However, when loading

images from the actual dataset (*IMAGE_DIR*), these subdirectory references aren't necessary and end up causing errors. To resolve this issue, the second line

```
df["image"] = df["image"].str.replace("image/", "", regex=False)
```

removes the “*image/*” prefix from all filenames in the “*image*” column. This is important because incorrect file paths might pop up when someone else tries to run the code and the model would be unable to locate images during training and inference. The *str.replace()* function operates on every row in the “*image*” column, replacing occurrences of “*image/*” with an empty string (“”). The argument *regex=False* makes sure that “*image/*” is treated as a string literal rather than a RegEx expression, preventing unintended replacements. Older versions of Pandas (earlier than 1.4.0) wouldn't have required this because *str.replace()* assumed a string literal by default, but with modern Pandas (1.4.0 and newer) *str.replace()* treats the first argument as a regex pattern. Since “*image/*” contains a forward slash, Pandas would misinterpret it as a RegEx metacharacter (Regex Tutorial, n.d.), and errors would ensue. This modification ensures that when images are loaded later using *os.path.join(IMAGE_DIR, filename)*, the file paths match the actual dataset structure. Without this adjustment, the program would attempt to load images from incorrect locations, especially when run outside of the dev environment. It's a small but important consideration to (hopefully) prevent file-not-found errors.

```
26 # Convert categorical class labels into integers
27 class_mapping = {label: idx for idx, label in enumerate(df["classes"].unique())}
28 df["label"] = df["classes"].map(class_mapping)
```

Encoding labels into numeric values. This section is responsible for converting categorical class labels (e.g., “scratch,” “dent,” “total_loss”) into numeric values so that the deep learning model can process them. This is accomplished by using a dictionary-based mapping system. First, the script extracts all unique class labels from the “*classes*” column using `df["classes"].unique()` to return a NumPy array containing all the unique values present in the DataFrame. It then applies `enumerate()` to assign a unique numeric indexed integer to each class, creating a dictionary where the keys are the original damage categories, and the values are their corresponding integer representations. This makes sure that every unique class is mapped systematically. Once the mapping dictionary is created, the script then applies it to the “*classes*” column using `df["label"] = df["classes"].map(class_mapping)`. This replaces each categorical label with its corresponding numeric index, storing the transformed values in a new “*label*” column. This is important because ML models can’t process raw text labels, they require numerical inputs for classification tasks (PyTorch, 2022). Encoding labels in this way also ensures consistency between training and inference. Since the same mapping is used throughout the project, there is no risk of mismatched labels leading to incorrect classifications when new images are analyzed. By systematically converting damage categories into numerical labels, VDRS gets the categorical data into the pipeline so that ResNet152 is able to effectively learn and differentiate between various types of vehicle damage.

```
30 # Print class mappings and distribution of damage labels
31 print("Class mapping:")
32 for class_name, idx in class_mapping.items():
33     print(f"{idx}: {class_name}")
```

Print class mappings and distribution of damage labels. VDRS prints the class mappings and label distribution toward the top of the output to provide the user with an idea about how the damage categories are laid out and the balance of the training data. The first line serves as a header for displaying the label encoding. Then, a *for* loop iterates through the *class_mapping* dictionary, which contains the key-value pairs where the key is the original class name (“scratch,” “dent”) and the value is the numeric index (0,1, 2, ...). Inside the loop, the formatted print statement *print(f"{idx}: {class_name}")* ensures that the output gets printed in a human-readable format. This method uses f-strings (*f”...”*) that allow for embedding variables and expressions inside curly brackets { } to allow for dynamic output formatting. The numeric index is represented by *{idx}*, and the category class label is represented by *{class_name}*.

```
Class mapping:
0: total_loss
1: head_lamp
2: door_scratch
3: no_damage
4: glass_shatter
5: tail_lamp
6: bumper_dent
7: door_dent
8: bumper_scratch
```

This process was helpful for debugging the class imbalance issues that were present in earlier versions of the program. If a class was missing, incorrect (the ‘unknown’ class), or incorrectly mapped, the printed output would help identify the issue before spending a lot of time training.

```
35     # Analyze the frequency distribution of class labels
36     class_counts = df["classes"].value_counts()
37     print("\nClass distribution:")
38     print(class_counts.to_string())
```

Analyze the frequency distribution of class labels. VDRS uses this block to output the distribution of class labels in the dataset, enabling an understanding of how the training dataset is imbalanced. The first line `class_counts = df["classes"].value_counts()` calculates the frequency of each unique class label in the “classes” column. The `value_counts()` function from Pandas returns a sorted count of occurrences for each class, allowing the user to quickly see how many samples belong to each category. This can be helpful to determine if certain damage types appear significantly more often than others, which could lead to biased predictions. The next line `print("\nClass distribution:")` is a simple print statement to serve as a header for the Class Distribution output, using `\n` to output a line break for nice formatting, separating this block of output from its predecessor and successor output blocks. The last line in this section `print(class_counts.to_string())`, outputs the class distribution, showing each damage category alongside its corresponding image count. Here, we append `.to_string()` to tell Pandas not to output the series object metadata (Name: count, dtype: int64) and instead just to convert the series into a formatted string representation without the extra metadata (Pandas Development Team, n.d.) This was just an aesthetic choice, as I thought that line of output looked sloppy. The result is a nice, clean output of the classes and their corresponding image counts.

```
Class distribution:
classes
no_damage      337
total_loss     210
door_dent      193
bumper_scratch 164
door_scratch   154
tail_lamp      137
glass_shatter  137
head_lamp      133
bumper_dent    129
```

```
40 # SciKit-Learn Train/Test Split
41 train_df, val_df = train_test_split(df, test_size=0.2, stratify=df["label"], random_state=13)
```

SciKit-Learn's *train_test_split()* function. VDRS uses SciKit-Learn's *train_test_split* function here to split the dataset into training and validation subsets while ensuring a fair distribution of class labels. The *train_df* DataFrame will become our training subset and the *val_df* DataFrame will become our validation subset. The *train_test_split()* function expects arguments in the parentheses for array(s), test size, train size, stratification of label distribution, and random state. The function also expects a parameter for *shuffle* to control whether the dataset is randomly shuffled before splitting into training and validation. As *shuffle=True* is the

default behaviour we don't need to explicitly include that argument. Since we're not using time-series data for this project and image order doesn't really matter, we're good to leave this as the default setting. The *test_size* argument is optimized at 0.2, meaning that 20% of the training dataset will be allocated to validation. We also don't need to explicitly specify that the remaining 80% of the dataset will be used for training, as SciKit-Learn automatically assigns the leftover portion when *test_size* is defined. The most important (and useful) part of this block is the argument for *stratify=df["label"]*, as it ensures that the distribution of class labels remains consistent across both the training and validation sets. This is a very cool feature that prevents the split from over/underrepresenting some classes, which could lead to biased training and inaccurate validation results. By stratifying based on the *"label"* column, we can maintain a balanced representation of damage categories, which is especially important for datasets with class imbalances. A *random_state* parameter is arbitrarily set to my lucky number 13 to provide a random seed so that the model remains consistent throughout the hyperparameter tuning process.

```
43 # Data Augmentation with torchvision
44 transform = transforms.Compose([
45     transforms.Resize((224, 224)),
46     transforms.RandomResizedCrop(224, scale=(0.8, 1.0)),
47     transforms.RandomHorizontalFlip(p=0.5),
48     transforms.RandomAffine(degrees=15, translate=(0.1, 0.1), scale=(0.9, 1.1), shear=10),
49     transforms.ColorJitter(brightness=0.3, contrast=0.3),
50     transforms.ToTensor(),
51     transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
52 ])
```

Data augmentation with TorchVision. VDRS employs advanced data augmentation techniques through TorchVision transforms to enhance a relatively limited training dataset, improving generalization and reducing overfitting. The LLM Grimoire (2025) helped me select appropriate transforms and set the value parameters for them. This block establishes the transform pipeline via ***transforms.compose()***, linking multiple image transformations applied during dataset preprocessing. The first transformation ***transforms.Resize((224, 224))*** ensures that all images are resized to 224x224 pixels to match ResNet152's model requirements. The Kaggle page where the data came from states that they already are (Manssor, n.d.), but it doesn't hurt to make sure. Next, ***transforms.RandomHorizontalFlip(p=0.5)*** applies a 50% probability of flipping an image horizontally to simulate different viewing angles. To introduce geometric transformations ***transforms.RandomAffine(degrees=15, translate=(0.1, 0.1), scale=(0.9, 1.1), shear=10)*** randomly rotates the image by up to 15 degrees, translates it by 10% along both axes, scales it between 90% and 110%, and applies a shear transformation of up to 10 degrees. These augmentations help the model be more adaptable to distortion and varying camera positions. Color-based augmentations use ***transforms.ColorJitter(brightness=0.3, contrast=0.3)*** to randomly adjust brightness and contrast +/- 30% to allow for the model to handle varying lighting transitions. The transformation ***transforms.ToTensor()*** converts the image from a PIL image to a PyTorch tensor, which is required for deep learning models in PyTorch (2024.) This conversion is important because PyTorch models expect inputs as tensors to enable efficient matrix operations and GPU acceleration. When an image is loaded as a PIL image, its pixel values are typically stored as unsigned 8-bit integers (0-255.) However PyTorch requires tensor values to be in floating point format (0.0-1.0) for numerical stability during training (PyTorch,

2024c). *ToTensor()* automatically scales pixel values by dividing each pixel by 255.0. This normalization step ensures that pixel intensities are consistent to improve model convergence. Finally, *transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])* normalizes pixel values using ImageNet's mean and standard deviation values, which were calculated from millions of images in the ImageNet dataset (Gravikov, 2022.) This normalization ensures that input images are on the same scale as those used to pre-train ResNet152, helping to maintain consistency in feature extraction and improve overall performance. Normalization is important for VDRS because deep learning models perform best when their input data has a zero-centered distribution with unit variance (Dhakal, 2024.) The mean values **0.485**, **0.456**, **0.406** correspond to the average pixel intensity for the RGB channels, and the standard deviation values of **0.229**, **0.224**, **0.225** help to scale the pixel intensities appropriately. This operation standardizes each pixel by subtracting the channel-wise mean and dividing by the channel-wise standard deviation (DataCamp, 2024) effectively transforming pixel values into a more stable range. Without normalization, the model might struggle with unbalanced pixel distributions, leading to longer training times and reduced accuracy. By applying ImageNet normalization, VDRS aligns its input images with the pre-trained ResNet152 model to allow for optimized feature extraction and classification performance.

```
55 # VehicleDamageDataset Class
56 class VehicleDamageDataset(Dataset):
57     def __init__(self, dataframe, transform=None):
58         self.dataframe = dataframe
59         self.transform = transform
60
61     def __len__(self):
62         return len(self.dataframe)
63
64     def __getitem__(self, idx):
65         img_path = os.path.join(IMAGE_DIR, self.dataframe.iloc[idx]["image"])
66         label = self.dataframe.iloc[idx]["label"]
67         image = Image.open(img_path).convert("RGB")
68         if self.transform:
69             image = self.transform(image)
70         return image, torch.tensor(label, dtype=torch.long)
```

The VehicleDamageDataset class. The VehicleDamageDataset class is a custom PyTorch Dataset designed to handle image loading and preprocessing from VDRS. It extends *torch.utils.data.Dataset*, making it compatible with PyTorch's DataLoader, which enables batch processing, shuffling, and parallel data loading for efficient model training. The class is structured with three methods, `__init__()`, `__len__()`, and `__getitem__()`, with each playing an important role in data retrieval and preprocessing. Grimoire (2025) helped me to find the right structure for this class. The `__init__()` method initializes the dataset by accepting a DataFrame containing image filenames and labels, along with an optional transform argument that allows for TorchVision-based preprocessing. The `__len__()` method returns the total number of samples in the dataset to make sure PyTorch's DataLoader can properly iterate through batches of data. The `__getitem__()` method is used to fetch an image and its corresponding label when given an index. It constructs the file path we set up earlier in the program using

os.path.join(IMAGE_DIR, filename), loads the image using Pillow's

Image.open(img_pth).convert("RGB"), and makes sure that it maintains a consistent color format. The label is extracted from the DataFrame, and if a transform pipeline is provided, it is applied to the image before returning it. Finally, the label is converted into a PyTorch tensor using *torch.tensor(label, dtype=torch.long)*. In PyTorch, *torch.long* (also called *torch.int64*) is a 64-bit integer data type, and it is useful for categorical labels in classification tasks because PyTorch's loss functions (like *CrossEntropyLoss*) expect class indices to be in integer format rather than floating-point numbers (Sling Academy, 2024.) The *VehicleDamageDataset* class is important to have set up correctly so that the image preprocessing pipeline can be automated, allowing PyTorch's *DataLoader* to efficiently manage data during training.

```
73     # Create datasets and the dataset autoloader
74     train_dataset = VehicleDamageDataset(train_df, transform=transform)
75     val_dataset = VehicleDamageDataset(val_df, transform=transform)
76     train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
77     val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False)
```

Creating datasets and the data autoloader. The dataset and dataloader setup in this block is important to feed data into VDRS during training and validation. The *train_dataset* and *val_dataset* are instances of the *VehicleDamageDataset* class. The *transform=transform* argument ensures that both the training and validation images undergo the same preprocessing

pipeline to maintain consistency with ResNet152's model input requirements. The PyTorch DataLoader is used to create *train_loader* and *val_loader*, which manage how data is fed into the model. The *batch_size=32* argument defines how many images are processed in a single forward pass, balancing computational efficiency with memory constraints. In *train_loader*, *shuffle=True* ensures that training samples are randomly shuffled before each epoch, preventing the model from memorizing the sequence of images and 'cheating' to improve generalization. However, in *val_loader*, *shuffle=False* ensures that validation samples are processed in a fixed order to allow for consistent performance evaluation without any unnecessary variation. By using DataLoader, VDRS benefits from efficient mini-batch processing, multi-threaded data loading, and optimized memory management, making it well-suited for large-scale deep learning training.

```
79      # Load ResNet152
80      from torchvision.models import ResNet152_Weights
81      model = models.resnet152(weights=ResNet152_Weights.DEFAULT)
82      num_ftns = model.fc.in_features
```

Loading the ResNet152 model. VDRS makes use of the pre-trained ResNet152 model from TorchVision for feature extraction and classification. The line

from torchvision.models import ResNet152_Weights imports the default weight configuration for the model, ensuring that the model is initialized with weights pre-trained on the ImageNet dataset rather than starting from a random initialization. The line

model = models.resnet152(weights=ResNet152_Weights.DEFAULT) ensures that the model is initialized with these pre-trained weights, allowing it to make use of features learned from its gigantic dataset. These pre-trained weights significantly improved the model's performance by providing a strong feature representation right out of the gate, reducing the amount of labeled training data required, and accelerating convergence during fine-tuning. The final line

num_ftrs = model.fc.in_features extracts the number of input features from ResNet152's fully connected (FC) layer. We will use this value later when we modify the classification head to match the number of damage categories in our dataset. By adapting the final layer while keeping the pre-trained convolutional layers frozen or just partially trainable, VDRS ensures that it retains the generalized visual feature extraction capabilities while still tailoring the model to the specific task of vehicle damage recognition.

```
84     # nn.Sequential and Dropout Layer
85     model.fc = nn.Sequential(
86         nn.Dropout(0.1), #10% seems to be optimum
87         nn.Linear(num_ftrs, len(class_mapping))
88     )
```

nn.Sequential and Dropout layer. VDRS modifies the FC layer of ResNet152's model to adapt it for vehicle damage recognition. Instead of using the default classification head, which was originally designed for ImageNet's 1,000 classes (Gravikov, 2022), this block replaces it with a custom classifier using **nn.Sequential()** which allows for multiple layers to be defined in sequence. The first component of the custom classifier is a dropout layer (**nn.Dropout(0.1)**), which applies a 10% dropout rate during training. Dropout layers help to prevent overfitting by randomly disabling 10% of the neurons in the fully connected layer per iteration, forcing the model to rely on a diverse set of learned features rather than memorizing specific patterns. This improves the model's ability to generalize with unseen images and reduces the risk of overfitting to the training data. Following the dropout layer, the **nn.Linear(num_fts, len(class_mapping))** layer serves as the final classifier, where **num_fts** represents the number of input features from ResNet152's last layer, and **len(class_mapping)** corresponds to the number of damage categories in VDRS. This linear layer takes the extracted features from ResNet and maps them to the correct damage class, enabling the model to make predictions specific to the listed vehicle damage types. By taking advantage of **nn.Sequential()**, VDRS ensures that these layers are logically structured and easily modifiable, making it straightforward to adjust dropout rates or classifier complexity as needed. The **nn.Sequential()** function is a container module in PyTorch (2024a) that allows multiple layers to be stacked together in a sequential order. It simplifies the model definition by automatically passing the output of one layer as the input to the next, eliminating the need to manually program forward propagation logic. It replaces ResNet's default FC layer **model.fc = nn.linear(2048, 1000)** where 2048 is the number of input features from ResNet's final convolutional layer, and 1,000 is the number of classes in ImageNet, which

is irrelevant for this vehicle damage classification project. By replacing *nn.linear* with *nn.sequential*, the model gains the flexibility to incorporate dropout regularization and a customized FC layer tailored to the specific number of vehicle damage categories. This modification allows the model to retain ResNet152's powerful feature extraction while specializing its classification capabilities for the VDRS dataset.

```
90     # Initially freeze all layers
91     for param in model.parameters():
92         param.requires_grad = False
```

Progressive unfreezing of layers. I used Claude's new model, Sonnet 3.7 (2025), to explore what adding a progressive unfreezing schedule would look like after my stable training model was already working. My goal was to see if an already well-performing model could be further improved in terms of validation accuracy and loss. However, the results showed no significant improvement compared to statically unfreezing layers at the start of execution. That said, it didn't degrade performance either, so I decided to keep it in the final submission as an interesting concept that I learned late in development. While this could be considered scope creep, it wasn't too disruptive to the project timeline. Now, VDRS has a moderately sophisticated progressive unfreezing schedule implemented, where all layers are initially frozen except for the fully connected (FC) layer, which remains trainable from the start. This ensures that early epoch training focuses solely on learning the classification-specific features while preserving the pre-trained feature extraction capabilities of the deeper convolutional layers. As training progresses and validation accuracy plateaus, additional layers are gradually unfrozen to allow the

model to fine-tune deeper representations of vehicle damage. PyTorch gives a boolean attribute ***requires_grad*** that determines whether a tensor's gradients should be computed during backpropagation. When ***requires_grad = False***, the corresponding parameter is frozen, meaning that it will not be updated during training. This ensures that the pre-trained weights will remain unchanged until explicitly unfrozen and carries the side benefits of avoiding unnecessary computation as well as reducing memory usage. Conversely, when ***requires_grad = True***, the parameter becomes trainable, allowing the optimizer to compute and apply gradients to adjust its values. In VDRS's progressive unfreezing strategy, all layers are initially frozen except for the FC layer. If validation accuracy fails to improve over x number of epochs (3 seems ideal), deeper layers of the network are incrementally unfrozen. This process allows the model to progressively adapt its feature extraction layers to the specific characteristics of vehicle damage while avoiding forgetting general image features learned during pre-training. The decision to unfreeze a layer is based on the number of consecutive epochs where validation accuracy did not improve over the best model saved. The progressive unfreezing schedule in VDRS follows a structure where layers are gradually unfrozen from deep to shallow, meaning that higher-level layers (closer to the output) are unfrozen first, while lower-level layers (closer to the input) remain frozen for longer.

```
94 # Define layers for progressive unfreezing - from deep to shallow
95 unfreezing_stages = [
96     model.layer4, # Unfreeze first layer
97     model.layer3, # Unfreeze second layer
98     model.layer2, # Unfreeze third layer
99     model.layer1, # Unfreeze first layer
100     model.bn1, # Unfreeze bn1
101     model.conv1 # Unfreeze conv1
102 ]
```

This approach makes sure that the most abstract, task-specific features, such as shapes and damage patterns, are fine-tuned early, while general image features like edges and textures remain stable until later in training. The deepest layers are responsible for learning complex representations that are highly specific to the dataset, so they are the first to be unfrozen when validation accuracy plateaus. This top-down unfreezing approach was meant to help balance stability and adaptability and prevent the model from diverging too quickly from the pre-trained weights while allowing for gradual specialization of the vehicle damage dataset.

```
104 # Unfreeze the FC layer and the first three layers at the start of training
105 unfrozen_idx = 3 # Start with layers 4, 3, and 2 unfrozen
106 unfreezing_stages.append(model.fc)
107 for i in range(unfrozen_idx + 1): # unfrozen_idx +1 to include model.fc
108     for param in unfreezing_stages[i].parameters():
109         param.requires_grad = True
110
111 # Move model to the appropriate device
112 model = model.to(device)
```

(Claude, 2025.) At the start of training, a subset of layers are made trainable right away. The variable *unfrozen_idx* = 3 determines how many layers should be immediately unfrozen from the *unfreezing_stages* list, which contains the ordered progression of layers to be trained. The line *model = model.to(device)* transfers the model to the hardware specified earlier in the code.

```

209     # Start to unfreeze layers after x epochs of no improvement
210     if no_improvement_count >= 3 and unfrozen_idx < len(unfreezing_stages):
211         print(f"No improvement for {no_improvement_count} epochs. Unfreezing next layer.")
212         for param in unfreezing_stages[unfrozen_idx].parameters():
213             param.requires_grad = True
214         unfrozen_idx += 1
215         no_improvement_count = 0 # Reset counter after unfreezing
216
217         # Re-create optimizer to properly register newly unfrozen parameters
218         optimizer = optim.Adam(filter(lambda p: p.requires_grad, model.parameters()), lr=new_lr)

```

(Claude, 2025) If no improvement is noted after x epochs and there are still unfrozen layers remaining, the model dynamically unfreezes the next layer in the *unfreezing_stages* list. The condition *no_improvement_count* $\geq x$ ensures that a new layer is only unfrozen after x continuous epochs without improvement over the best saved model to prevent unfreezing new layers while the model is still learning effectively. The condition

unfrozen_idx $< \text{len}(\text{unfreezing_stages})$ ensures that the script doesn't attempt to unfreeze layers beyond the number of available trainable layers. When these conditions are met, the script prints a message that no improvement was observed, and the next layer in the sequence is now being unfrozen.

```

Epoch 22: Training Loss = 0.0358, Training Accuracy = 98.67%
Epoch 22: Validation Loss = 0.5395, Validation Accuracy = 87.15%, Learning Rate = 0.00003000
No improvement for 3 epochs. Unfreezing next layer.

```

The parameters of this layer are then updated by setting *requires_grad* = *True*, which allows it to be trained, *unfrozen_idx* is incremented, and *no_improvement_count* is reset to zero. Since PyTorch optimizers don't automatically recognize newly unfrozen layers, the optimizer is recreated after each unfreezing event to properly register the updated trainable parameters. The

optimizer only includes parameters where *requires_grad = True*, ensuring that frozen layers remain untouched while newly unfrozen layers are included in gradient calculations.

```

115 # **Calculate Class Weights for Imbalanced Dataset**
116 class_weights = compute_class_weight(
117     class_weight='balanced',
118     classes=np.unique(train_df['label']),
119     y=train_df['label'].values
120 )
121 class_weights = torch.tensor(class_weights, dtype=torch.float).to(device)
122 print("\nClass weights:")
123 for idx, weight in enumerate(class_weights):
124     class_name = [k for k, v in class_mapping.items() if v == idx][0]
125     print(f"{class_name}: {weight.item():.4f}")

```

Weights for class imbalance in the dataset. This section defines and displays class weights to handle the class imbalance inherent in the dataset. Grimoire (2025) helped me identify the need for this and propose a solution. I did not modify the LLM's code at all. The process begins by computing class weights using *compute_class_weight()* from SciKit-Learn's library. Class imbalance is present in my dataset and correction is needed to avoid bias when damage categories appear more or less frequently than others. By assigning higher weights to underrepresented categories, the function ensures that the model pays more attention to these categories during training. The parameter *class_weights='balanced'* instructs *compute_class_weight()* to automatically assign weights inversely proportional to class frequencies. The parameter *classes=np.unique(train_df['label'])*, extracts the unique class labels present in the training dataset. The parameter *y=train_df['label'].values* provides the actual class labels as a NumPy array, which *compute_class_weight()* uses to determine frequency distributions. Once the class weights are computed, they are converted into a PyTorch tensor using *class_weights = torch.tensor(class_weights, dtype=torch.float).to(device)*.

The `dtype=torch.float` ensures that the weights are stored as floating-point numbers, which are required during loss computation. The `.to(device)` ensures that class weights are being stored on the same hardware that the model is using to avoid mismatches during training. The next block prints the class weights to output in a human-readable format.

```
Class weights:
total_loss: 0.8433
head_lamp: 1.3365
door_scratch: 1.1518
no_damage: 0.5247
glass_shatter: 1.2879
tail_lamp: 1.2879
bumper_dent: 1.3754
door_dent: 0.9199
bumper_scratch: 1.0814
```

The `for` loop here iterates over `enumerate(class_weights)`, where `idx` represents the class index and `weight` is the corresponding computed class weight for that index. Since the class weights use numerical labels, the model requires a method to retrieve the human-readable class names (i.e., “no damage” or “total loss.”) To achieve this, the line

`class_name = [k for k, v in class_mapping.items() if v == idx][0]` performs a dictionary lookup in `class_mapping`, which contains class names as keys and their assigned numerical labels as values. The list comprehension loop iterates through each key-value pair, selecting the key `k` whose value `v` matches `idx`. Since there is only one matching key per index, `[0]` extracts the first (and only) element from the resulting list. This mapping allows the model to print class names instead of numeric labels, making it easier to interpret the assigned class weights. It works by updating the model’s parameters based on the gradients computed during backpropagation. The

model.parameters() argument ensures that all trainable weights in the model are updated during optimization.

```
127 # Define Loss Function with Class Weighting
128 criterion = nn.CrossEntropyLoss(weight=class_weights)
129 optimizer = optim.Adam(model.parameters(), lr=0.00003)
```

Cross-entropy loss function and optimizer. These lines define the loss function and optimizer for training VDRS. The loss function, *nn.CrossEntropyLoss()*, measures the difference between the predicted class probabilities and the actual class labels. Given the dataset's imbalance, we will use the *weight=class_weights* parameter to assign higher penalties to underrepresented classes, avoiding bias toward overrepresented damage categories. The optimizer employs *optim.Adam()*, sourced from PyTorch's suite of optimization algorithms, to update model parameters during training. The learning rate is set to a very small value ($3 * 10^{-5}$) to facilitate stable and gradual training, preventing the model from making large, unstable weight updates. This is especially important when using a pre-trained model like ResNet152, as a high learning rate could overwrite useful features learned from ImageNet. For VDRS, *nn.CrossEntropyLoss()* is the most suitable choice because the task involves multi-class classification where the model predicts one discrete class from several categories. Mean squared error is inappropriate since this isn't a regression model, and binary cross-entropy doesn't apply because we're not classifying damaged vs. not damaged. Our training dataset isn't imbalanced enough to warrant a focal loss solution, and we're not utilizing a margin-based classification system like a support vector machine, so a hinge loss setup is also not relevant. For VDRS, cross-entropy is advantageous as it applies a log softmax internally, eliminating the need to

manually apply a softmax activation function to the model's outputs. A cross-entropy loss function effectively penalizes incorrect confident predictions and offers the additional benefit of supporting class weighting, which is essential for VDRS.

```
131 # Dynamic Learning Rate Scheduler
132 scheduler = optim.lr_scheduler.ReduceLROnPlateau(
133     optimizer, mode="min", patience=1, factor=0.5, min_lr=1e-6
134 )
```

Dynamic learning rate scheduler. The dynamic learning rate scheduler in VDRS utilizes PyTorch's *optim.lr_scheduler.ReduceLROnPlateau()*, which monitors the validation loss and adjusts the learning rate when it detects a plateau. Since deep learning models often encounter plateaus in performance during training, dynamically adjusting the learning rate enables the optimizer to escape local minima and fine-tune model weights more effectively. PyTorch offers several learning rate scheduling modules, and VDRS employs ReduceLROnPlateau (RLROP). RLROP is superior to other learning rate scheduling systems like StepLR (which reduces the learning rate by a fixed factor at predetermined intervals) or CosineAnnealingLR (which gradually reduces the learning rate to a minimum value over a set cycle) because it adopts a more adaptive strategy based on validation performance rather than relying on fixed schedules. RLROP intelligently monitors the validation loss and responds dynamically when the model stops improving. This makes RLROP particularly effective in situations where the training process is unpredictable, as it allows the optimizer to focus on fine-tuning only when necessary. As we are already starting with a very gradual learning rate due

to using ResNet152's pre-trained weights, RLROP ensures that learning rate reductions only occur when performance gains plateau. RLROP takes five arguments in this program. After initializing the scheduler, the *optimizer* argument needs to be specified in order to access *optimizer.param_groups* to be able to adjust the learning rate. The *mode="min"* argument tells the scheduler to track a metric that should be minimized, in this case, the validation loss. If we wanted to track validation accuracy, we would have used *mode="max."* The *patience=1* argument determines how many epochs the scheduler will wait after detecting no improvement before reducing the learning rate. I had originally set this to *patience=3* but noticed that the learning rate never changed throughout all 50 epochs. I ended up setting *patience=1* to diagnose the issue, but it ended up working perfectly at this configuration, so that's where I kept it. The *factor=0.5* argument determines how much the learning rate is scaled down when triggered. In this case, the learning rate is reduced by half every time the scheduler determines to reduce it. The learning rate keeps getting halved until it reaches the lower bound for how small the learning rate can become. This is configured using *min_lr=1e-6* to denote the minimum learning rate of 1×10^{-6} or 0.000001, which prevents the optimizer from stalling completely.

```
184     val_accuracy = 100 * val_correct / val_total
185     old_lr = optimizer.param_groups[0]["lr"] # Store the current learning rate before the step
186     scheduler.step(val_loss / len(val_loader))
187     new_lr = optimizer.param_groups[0]["lr"] # Get the updated learning rate if it changed
```

During each validation step inside the training loop, the learning rate scheduler checks whether to adjust the learning rate based on the validation loss. The process begins by calculating the validation accuracy using *val_accuracy = 100 * val_correct / val_total*, which calculates the percentage of correctly predicted samples in the dataset. Next, the current learning rate is stored

in `old_lr = optimizer.param_groups[0]["lr"]`, ensuring any changes to the learning rate can be detected after the scheduler step. We can see how this works as PyTorch is using *optim.Adam* to manage parameters using parameter groups. The object *optimizer.param_groups* is a list of dictionaries, where each dictionary contains settings for a specific group of model parameters. If an optimizer has only one parameter group (which is common), its settings are stored in *optimizer.param_groups[0]*. Therefore, *optimizer.param_groups[0]["lr"]* accesses the key within the dictionary, which holds the current learning rate for that parameter group. Next, the scheduler is updated using *scheduler.step(val_loss / len(val_loader))* to take the average validation loss over the entire validation dataset to determine whether the learning rate should be reduced. Here, I had initially (incorrectly) been using *scheduler.step(val_loss)*, which didn't give any fatal errors, but it was this line that was preventing the learning rate from decreasing when plateaus were reached in validation loss. Claude (2025) pointed out that I needed to be passing the average validation loss, instead of the total validation loss, which cleared up the issue. Finally, `new_lr = optimizer.param_groups[0]["lr"]` retrieves the updated learning rate, allowing the code to compare it to `old_lr`.

```

192     #Print update to learning rate
193     if new_lr < old_lr:
194         print(f"~~~Learning rate decreased: {old_lr:.8f} → {new_lr:.8f}~~~")
195
196     # Print epoch statistics
197     new_lr = optimizer.param_groups[0]["lr"]
198     print(
199         f"\nEpoch {epoch + 1}: Training Loss = {running_loss / len(train_loader):.4f}, Training Accuracy = {train_accuracy:.2f}%"
200     )
201     print(
202         f"Epoch {epoch + 1}: Validation Loss = {val_loss / len(val_loader):.4f}, Validation Accuracy = {val_accuracy:.2f}%, Learning Rate = {new_lr:.8f}"

```

If the learning rate has decreased, an output is printed that the optimizer has updated its learning rate. The check is performed using *if new_lr < old_lr*, ensuring that an update is only displayed when a reduction occurs. I added tildes (~~~) to make these lines stand out as learning rate

updates. Additionally, the learning rate is printed to the output of the epoch statistics that summarize training progress. If I have time, I may try to make something graphical in Matplotlib that displays learning rate changes as the epochs progress.

```
136 # Training Loop Variables
137 num_epochs = 50
138 best_accuracy = 0.0
139 best_loss = float("inf")
140
141 # Progressive Unfreezing Variables
142 no_improvement_count = 0
143 previous_best = 0.0
```

Training and progressive unfreezing variables list. This section of the code initializes two groups of variables for the main training loop and the progressive unfreezing system. Here, *num_epochs = 50* defines the number of training iterations over the dataset. If you want to shorten the training duration, this is the only variable you need to adjust. However, be aware that some features that emerge in later epochs may not have had time to meet the requirements to trigger them. The variable *best_accuracy = 0.0* is initialized here to track the highest validation accuracy achieved during the training loops. The *best_loss = float("inf")* variable tracks the lowest validation loss encountered during the training loop to ensure that the model checkpointing systems only save the best-performing model that will later be used by the damage identifier program. This value gets initialized to infinity so that any real validation loss calculated in the first epoch will be guaranteed to be smaller, ensuring an immediate update when training begins. The progressive unfreezing variables are important for implementing the strategy, where deeper layers of the model are gradually unfrozen as training progresses. The

variable *no_improvement_count* = 0 is initialized to count the number of consecutive epochs where validation accuracy does not improve. This variable helps determine when layers should be unfrozen. The variable *previous_best* = 0.0 stores the highest validation accuracy achieved so far. If the validation accuracy does not surpass *previous_best* for a set number of epochs, a new layer is unfrozen to allow further fine-tuning.

```
145     # Main Training Loop
146     for epoch in range(num_epochs):
147         # Training phase
148         model.train()
149         running_loss = 0.0
150         train_correct = 0
151         train_total = 0
```

Beginning of the main training loop. The main training loop begins with an outer *for* loop that iterates over the number of epochs specified by *num_epochs*. Within each epoch, the model enters training mode using *model.train()*, which ensures that different layers, like the dropout layer and batch normalization, run appropriately during training. Several other tracking variables are initialized here. The *running_loss* = 0.0 variable is initialized and will accumulate the total training loss for each epoch. This variable gets updated during the phase where the model makes predictions on the training dataset, computes the loss, and adjusts weights accordingly. The variable *train_correct* = 0 tracks the number of correctly predicted training samples. The variable *train_total* = 0 tracks the total number of samples processed. These variables are reset at the start of each epoch to ensure that metrics are computed independently for each training iteration.

```
153     # Inner for Loop
154     for images, labels in train_loader:
155         images, labels = images.to(device), labels.to(device)
156         optimizer.zero_grad()
157         outputs = model(images)
158         loss = criterion(outputs, labels)
159         loss.backward()
160         optimizer.step()
161         running_loss += loss.item()
162
163     # Calculate training accuracy
164     _, predicted = torch.max(outputs, 1)
165     train_total += labels.size(0)
166     train_correct += (predicted == labels).sum().item()
```

The middle *for* loop. This section of the training loop processes batches of training data, computes the loss, updates the model's weights, and tracks training accuracy. The middle loop iterates over *train_loader*, which contains mini-batches of training images and their corresponding labels. First, *images* and *labels* are transferred to the designated hardware (CPU or GPU) using *.to(device)*, this ensures that all operations on the selected hardware device. The optimizer's gradients are then reset with *optimizer.zero_grad()*. to prevent accumulation from previous iterations. The model then processes the input batch, generating predictions with *outputs = model(images)*, and calculates the loss using *criterion(outputs, labels)*, where *criterion* was previously defined as *nn.CrossEntropyLoss(weight=class_weights)*, measuring how different the predictions are from the actual labels. The loss is then backpropagated with *loss_backward()*, computing gradients for each parameter in the model. The *optimizer.step()* function updates the model weights based on these gradients, applying the *Adam* optimization algorithm to minimize loss, as it was defined earlier with

optimizer = optim.Adam(model.parameters(), lr=0.00003). The *running_loss* variable is updated using *running_loss += loss.item()*, accumulating the total loss for the epoch. This allows the program to later compute the average training loss across all batches. For accuracy calculations, *torch.max(outputs, 1)* selects the highest probability class from the model's output for each sample in the batch, storing the predicted labels. To increment the total count of training samples processed, the incrementer *train_total += labels.size(0)* is used. The number of correctly predicted labels is determined by comparing *predicted* to *labels* and summing the matches using *(predicted == labels).sum().item()*, which is then accumulated in *train_correct*.

The middle *for* loop is doing a lot of work here and can reasonably be described as the core of the training process. It handles the forward pass, loss computation, backpropagation, optimizer updates, and accuracy tracking for each batch of data. Each mini-batch from *train_loader* is processed sequentially, ensuring that the model's weights are gradually adjusted based on its performance. Without *optimizer.zero_grad()*, previous gradient accumulation from previous iterations would cause problems and lead to incorrect updates. The forward pass is where the model generates predictions for the current batch, and the loss function measures how far the predictions are from the actual labels. Backpropagation is using *loss.backward()* from PyTorch to compute the gradients for all the model's parameters, and we don't have to worry about the complex mathematical computations taking place behind the curtain, so there's no need to manually derive and implement the partial derivatives or integral statements for gradient calculations. This level of abstraction makes training deep learning models much easier than manually implementing backpropagation, as frameworks like PyTorch handle all the low-level math while allowing the developer to focus on designing and training the model. Adam

optimizer updates all the parameters based on these gradients, automatically adjusting the model's weights in a direction that reduces the loss. The ***running_loss*** variable accumulates the total loss across all batches, allowing the model to later compute the average loss per each epoch. The middle ***for*** loop ensures that each training sample contributes to refining the model's learned representations, with the optimizer adjusting weights incrementally to minimize classification error.

```
170     # Validation phase variables
171     model.eval()
172     val_correct = 0
173     val_total = 0
174     val_loss = 0.0
```

The validation phase variables. The validation phase begins with ***model.eval()***, which sets the model into evaluation mode in PyTorch. This mode is important because it affects certain layers of the model, particularly batch normalization and dropout, ensuring they perform correctly during inference by using their learned statistics rather than updating them as they would in the training phase. Once the model is set to evaluation mode, three key tracking variables are initialized. The ***val_correct*** variable is set to zero and will count the number of correctly predicted validation samples. The ***val_total*** variable also gets set to zero and will track the total number of samples processed in the validation dataset. Finally, ***val_loss*** is initialized to zero, which will accumulate the total validation loss throughout the evaluation phase. Since no

weight updates occur during validation, this loss is purely used as a metric to assess the model's generalization performance.

```
176     # Inner for Loop
177     with torch.no_grad():
178         for images, labels in val_loader:
179             images, labels = images.to(device), labels.to(device)
180             outputs = model(images)
181             loss = criterion(outputs, labels)
182             val_loss += loss.item()
183             _, predicted = torch.max(outputs, 1)
184             val_total += labels.size(0)
185             val_correct += (predicted == labels).sum().item()
```

The inner *for* loop. The inner *for* loop in the validation phase processes the validation dataset by batch, ensuring that predictions are made without updating the model's parameters. The loop begins inside a *with* statement that disables gradient computation. The statement *with torch.no_grad():* ensures that no gradients are calculated during validation, which serves to improve efficiency and reduce memory usage since gradient tracking is unnecessary when the model is not actively being trained. Inside the loop, the validation data is iterated over using mini-batches by executing the line *for images, labels in val_loader:*. Here, *val_loader* is our PyTorch DataLoader object that loads batches of validation images and their corresponding labels. Each batch is then transferred to the previously-selected hardware using *images, labels = images.to(device), labels.to(device)* to give consistency across computations. The model then performs forward propagation by passing the batch of images through the

network using *outputs = model(images)*. This generates raw class scores that represent the model's confidence scores for each class. The loss is then calculated using

loss = criterion(outputs, labels) where we have already defined *criterion* as the *nn.CrossEntropyLoss(weight=class_weights)* function. This measures how different the predicted class values are from the actual class labels. The computed loss is incremented using *val_loss += loss.item()*, which enables the program to calculate the total validation loss across all batches. Next, the model's predictions are extracted using *_, predicted = torch.max(outputs, 1)*, which selects the class with the highest probability for each image in the batch. The total number of processed validation samples is updated with *val_total += labels.size(0)*, which ensures that all validation samples are counted. Finally, the number of correctly predicted samples is calculated with *val_correct += (predicted == labels).sum().item()* which compares the model's predictions to the actual labels, sums up the number of correct predictions, and adds them to *val_correct*. Since the entire process takes place within *torch.no_grad()*, no backpropagation occurs, and the model parameters remain unchanged. This ensures that validation provides an unbiased evaluation of the model's performance without inadvertently influencing learned weights.

```
203     # Save Best Model Checkpoint
204     if val_accuracy > best_accuracy or (val_accuracy >= best_accuracy - 0.5 and val_loss < best_loss):
205         best_accuracy = val_accuracy
206         best_loss = val_loss
207         checkpoint = {
208             'model_state_dict': model.state_dict(),
209             'optimizer_state_dict': optimizer.state_dict(),
210             'class_mapping': class_mapping # Save class mappings
211         }
212         torch.save(checkpoint, "best_resnet152_vehicle_damage_candidate.pth")
213         print(f"!!!New best model saved. Accuracy: {best_accuracy:.2f}%, Loss: {best_loss:.4f}!!!")
214         no_improvement_count = 0 # Reset counter when improvement happens
215     else:
216         no_improvement_count += 1 # Increment counter when no improvement
```

The best model checkpoint. The model checkpointing system in the training loop ensures that the best-performing model is saved based on validation accuracy and validation loss. The logic is based on a relatively sophisticated *if or* statement that checks whether the model's validation accuracy has exceeded the previous best accuracy or if the validation accuracy is within 0.5% of the best accuracy but achieves a lower validation loss. This condition allows the program to save models that might have slightly lower accuracy but better generalization, as indicated by a lower loss. If either condition is met, *best_accuracy* and *best_loss* are updated to reflect the new values. A dictionary called *checkpoint* is created, storing three key elements. The model's learned parameters are stored in *model_state_dict*. The optimizer's current state, including momentums (from Adam optimizer) and learning rates, is saved in *optimizer_state_dict*, making sure that training can resume seamlessly if needed. Additionally, *class_mapping* is stored within the checkpoint, preserving the mapping between class labels and numerical indices to allow for correct interpretations when loading the model separately from the trainer. I've been running this model a lot, and when a good model gets saved, I rename the file from *best_resnet152_vehicle_damage_candidate.pth* to *best_resnet152_vehicle_damage.pth*,

which will be called on in a separate inference program where a user can upload their own unseen photos for classification. To provide user feedback while the model is running, a formatted print statement outputs a line after the epoch information showing the new best model's statistics and indicating that the new optimal model has been recorded as the candidate file. Conversely, if the model does not meet the conditions for improvement, the *else* block increments *no_improvement_count* by 1, tracking consecutive epochs without meaningful progress for the progressive learning rate and progressive unfreezing functions.

Interactive Damage Predictor

The Interactive Damage Predictor (IDP) is a Python application that allows users to classify vehicle damage in an image using the model and weights created within the Vehicle Damage Recognition System (VDRS.) Unlike the training phase, this program works with new, unseen images that were not part of the original training dataset. Users can select any vehicle damage image from the web or use their own photos so long as it corresponds to one of the damage categories the model recognizes, and see how well the system is able to classify it. The program starts by prompting the user to enter a file path of an image. The image undergoes preprocessing (resizing, normalization, and tensor conversion) before being analyzed by a ResNet152-powered convolutional neural network. The model will predict the type of damage from one of the predefined categories (total loss, headlamp, door scratch, no damage, glass shattered, tail lamp, bumper dent, door dent, or bumper scratch) and provide a confidence score, indicating how certain the model is about its classification. To improve usability, the program includes an error handler for common input mistakes, such as removing unnecessary quotation marks when users copy and paste file paths from Windows' right-click "copy as path" feature.

After prediction, the user can either classify another image or exit the program, making it convenient for testing multiple images in one session. By allowing users to try the model on their own images and those sourced from the web, IDP provides a practical way to test and validate the model's accuracy in real-world scenarios. GPU acceleration is utilized (when available), ensuring fast and efficient processing. Let's take a look 'under the hood'!

IDP Libraries and Tools

The import section of the IDP script brings back four familiar libraries that enable deep learning-based image classification.

```
1 import torch
2 import torchvision.transforms as transforms
3 from torchvision.models import resnet152
4 from PIL import Image
```

PyTorch once again serves as the core deep learning framework, providing the tools necessary for loading the trained model, performing inference, and handling tensor operations efficiently. It enables seamless integration with GPU acceleration, ensuring that the classification process runs as quickly and efficiently as possible when compatible hardware is available. PyTorch is particularly useful for deep learning applications due to its extensive library support. TorchVision is one such library that plays an important role in preprocessing input images to match the format expected by the model. ResNet152, imported from *torchvision.models* acts as the backbone of the classification system. This is a deep convolutional neural network known for its high accuracy in image recognition tasks. It was originally pre-trained on large-scale datasets and fine-tuned by the VDRS training program. This allows IDP to take advantage of powerful feature

extraction capabilities, making it highly effective at distinguishing between the different types of vehicle damage. Finally, the Python Imaging Library (PIL) is imported via *Image* from *PIL*. It is used to handle image loading and basic manipulation. Since images come in various formats and color modalities, PIL ensures that they are opened correctly and converted to RGB format before processing. This step prevents compatibility issues and guarantees that every image is properly prepared for classification. Together, these libraries form the foundation of IDP, allowing it to efficiently process images, extract meaningful features, and accurately classify vehicle damage based on its pre-trained model. Now that we understand the toolset, let's move on to how the model itself is loaded and prepared for inference.

Setting the model path and selecting hardware. The next section of IDP is responsible for setting the path for the best-trained model from VDRS and determining what processor the classifier will run on. The script first defines *model_path*, specifying the location of the pre-trained model weights files *best_resnet152_vehicle_damage.pth*. This file was created from the highest scoring over several days of running the training model by renaming the *best_resnet_152_vehicle_damage_candidate.pth* file after an exceptionally high-scoring (lucky) training cycle, scoring 90.91% validation accuracy.

```
Class-wise validation accuracy:
total_loss: 90.48% (38/42.0)
head_lamp: 81.48% (22/27.0)
door_scratch: 90.32% (28/31.0)
no_damage: 88.06% (59/67.0)
glass_shatter: 100.00% (27/27.0)
tail_lamp: 92.59% (25/27.0)
bumper_dent: 76.92% (20/26.0)
door_dent: 84.62% (33/39.0)
bumper_scratch: 90.91% (30/33.0)

Best validation accuracy: 90.91%
Press Enter to exit...
```

The best-trained.pth file is included alongside each program so the end-user doesn't have to spend any more time than they would like running the computationally intensive training program and can jump straight to the IDP inference program using these weights. Next, the script dynamically sets the computing device by checking whether or not a CUDA-enabled GPU is available. If a compatible GPU is detected, the script assigns *cuda* to the *device* variable, enabling GPU acceleration for significantly faster inference. If no GPU is available, the model will run on the CPU instead, ensuring compatibility with a wide range of hardware configurations. This setup ensures that IDP can run as efficiently as possible, using GPU hardware if it's available while maintaining functionality on systems without one.

```
# Load the checkpoint
checkpoint = torch.load(model_path, map_location=device, weights_only=True)
```

Loading the model checkpoint. The next step in IDP involves loading the model checkpoint, which contains the trained weights required for vehicle damage classification. This is done using *torch.load()*, which reads the model file specified in *model_path* and loads it into memory. The *map_location=device* argument ensures that the model is loaded onto the appropriate hardware. Additionally, *weights_only=True* is set to allow PyTorch to only load the model weights, excluding unnecessary metadata and optimizer states. This makes the loading process more memory efficient as only the weights are needed for inference.

```
13 # Load class mapping from checkpoint
14 class_mapping = checkpoint.get('class_mapping', {
15     0: "total_loss",
16     1: "head_lamp",
17     2: "door_scratch",
18     3: "no_damage",
19     4: "glass_shatter",
20     5: "tail_lamp",
21     6: "bumper_dent",
22     7: "door_dent",
23     8: "bumper_scratch"
24 })
```

Mapping numerical categories to text. After loading the model checkpoint, IDP retrieves the class mappings, which define how the model's numerical predictions correspond to human-readable damage categories. This mapping gets extracted using *checkpoint.get('class_mapping', {...})* to ensure that if the class mapping is somehow missing from the checkpoint, a default mapping is available as a failsafe. The class mappings are self-explanatory. The list of categories is somewhat limited by the labels assigned to the 1,512 training images found on Kaggle (Manssor, n.d.). Future versions of VDRS and IDP would ideally have a much more versatile set of categories since real-world damages only rarely fall into these classes and oftentimes overlap, properly belonging to one or more categories. This is a known weakness of this program, but it is robust enough to demonstrate mastery of the course curriculum. Should a more versatile training dataset become available, re-mapping damage classes would be trivially easy.

```
26 # Class Mapping Hotfix - Swaps the key:values from model weights to output  
27 class_mapping = {v: k for k, v in class_mapping.items()}
```

Class mapping key:value swap. This line exists because class mappings in model checkpoints are stored with their numerical indices as keys instead of category names. Since the model outputs a numerical index as its prediction, the IDP script needs to swap this mapping to correctly translate the predicted index into human-readable damage categories. Without this fix, the program would return numerical outputs that, while technically correct, are visually unappealing. The issue originates from the way that class mappings are saved during training. Ideally, I would revise the VDRS file to store the class mappings in the correct format to eliminate the need for this line, but with the project deadline quickly approaching, I am hesitant to edit the working code when a simple swap line in IDP provides a working solution. This line works by iterating over the *class_mapping* dictionary with a *for* loop

{v: k for k, v in class_mapping.items()} that extracts each key:value pair where the key represents the numerical index and the value represents the category name. It swaps these so that the category name becomes the key and the numerical index becomes the value. This transformation allows IDP to correctly output the model's predicted numerical index as a human-readable damage category.

```
29 # Load ResNet-152 Without the Pre-Trained Weights
30 model = resnet152(weights=None)
31 num_fts = model.fc.in_features
```

Loading ResNet152 architecture. This section of the script loads the ResNet152 architecture. Since we created our own custom weights using VDRS, this implementation sets *weights=None* to prevent the model from loading ResNet’s default weights from ImageNet. Rather, the script will load the customized weights from the checkpoint file, ensuring that the model is customized for vehicle damage recognition rather than a general classifier. The line *num_fts = model.fc.in_features* extracts the number of input features in the fully connected (FC) layer of ResNet152. This is necessary to correctly define the final classification layer, ensuring it matches the number of damage categories in IDP. Without this adjustment, the model’s architecture would not align with the fine-tuned structure used during training in VDRS. By initializing ResNet in this way, the script ensures that the model retains the powerful feature extraction ResNet provides, while using our own custom-trained parameters tailored specifically for vehicle damage recognition.

```
33     # Ensure model architecture matches training
34     model.fc = torch.nn.Sequential(
35         torch.nn.Dropout(0.1), # Keep dropout as in training
36         torch.nn.Linear(num_fts, len(class_mapping))
37     )
```

Customizing the FC layer for vehicle damage. This section of the script modifies the FC layer of ResNet to match the architecture used during the training of VDRS. Since ResNet was designed for general image classification with 1,000 output classes through ImageNet, it must be customized to classify damage into the nine predefined categories represented in the training data. To achieve this, the script replaces the FC layer with a new sequential layer using *torch.nn.Sequential()*. The first component *torch.nn.Dropout(0.1)* applies dropout regularization with a 10% probability a neuron is dropped out to prevent overfitting. This ensures that neurons randomly deactivate during training, improving the model's generalization ability. The dropout remains the same as in VDRS to maintain consistency. The second component *torch.nn.Linear(num_fts, len(class_mapping))* defines an FC layer that takes *num_fts* as the number of input features from ResNet, and outputs *len(class_mapping)* neurons, where each neuron corresponds to one of the damage categories. This ensures that the model produces the correct number of output classes when performing inference. By modifying the final classification layer, this section of the script aligns the ResNet model with the custom training configuration used in VDRS, allowing it to correctly classify vehicle damage.

```
39 # Load the pre-trained weights
40 model.load_state_dict(checkpoint['model_state_dict'])
41
42 # Move model to the hardware device and set to evaluation mode
43 model = model.to(device)
44 model.eval()
```

Loading pre-trained weights and enabling evaluation mode. This section of the script loads the pre-trained weights from the checkpoint file and prepares the ResNet model for inference. The first line, `model.load_state_dict(checkpoint['model_state_dict'])`, restores the model's weight parameters from the checkpoint file. The weights were fine-tuned during training using VDRS and, once loaded, are ready for inference. The model is transferred to the appropriate hardware using `model.to(device)`, and `model.eval()` sets the model to evaluation mode. Evaluation mode disables certain training-specific behaviors, such as dropout and batch normalization updates. You may wonder, why we set dropout to 0.1, only to disable it a few lines later. We needed to include the dropout layer in `model.fc = torch.nn.Sequential()` because it needs to be present in the model architecture both during training and inference so that the structure remains consistent with the version used in training. The pre-trained weights from VDRS were tuned using a model that included dropout, meaning that when we load the checkpoint file, PyTorch expects the architecture to match exactly. If we removed dropout, the structure of `model.fc` would be different than the saved model and would cause an error when trying to load the weights. Including dropout ensures that if we ever decide to retrain the model in the future, the dropout layer will still be available. When switching back to `model.train()`, dropout will automatically reactivate. This is a standard practice (Latope2-150, 2018) in deep learning architecture as it ensures the model remains structurally consistent across both training

and inference phases. With the model now fully loaded, optimized for hardware, and set to evaluation mode, it's ready to process and classify user-submitted images for vehicle damage classification.

The image preprocessing pipeline. This section of the script defines the image preprocessing steps that allow for consistent results across a wide variety of user-submitted images. The transformations are applied using TorchVision's *transforms* module, which works alongside PIL to process user-submitted images. These steps include resizing the images to 224x224 pixels, the standard input size for ResNet152. This ensures consistency across all images, as deep learning models expect fixed input dimensions. Next, *transforms.ToTensor()* converts the images from Python's PIL format into a PyTorch tensor. PyTorch requires tensors as inputs rather than raw image data. Additionally, this conversion scales pixel values from 0-255 integer range to a 0-1 floating point range, making the data more suitable for deep-learning calculations. The final transform applies mean and standard deviation normalization using *transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])* based on ImageNet's dataset statistics. Normalizing the images using these values ensures that the input distribution is similar to the training distribution, improving classification accuracy. This normalization step standardizes pixel intensity values, helping the model generalize better across different lighting conditions and image variations. IDP should be able to make predictions on varying image file types, supporting .jpg, .jpeg, .png, .bmp, .gif, .tiff, and other common formats. By applying the same preprocessing steps as in VDRS other than the data augmentation

transformations, IDP makes sure that every input image is properly formatted, scaled, and normalized (Stewart, 2023) to enable the classifier to make accurate and consistent predictions.

```
53 # Define the image loading function
54 def predict_damage(image_path):
55     try:
56         image = Image.open(image_path).convert("RGB")
57     except Exception as e:
58         print(f"Error loading image: {e}")
59     return
```

Defining the image loading function. This section of the script defines the *predict_damage(image_path)* function, which is responsible for loading an image file and preparing it for classification. The architecture uses *try* and *except* to gracefully handle any path errors when opening the image. Inside the *try* block, *Image.open(image_path).convert("RGB")* is used to open the image using PIL and convert it to RGB format to ensure that the image will have exactly three color channels (red, green, and blue), which is necessary because this is what ResNet expects. Without this step, greyscale, CMYK, or images with an alpha channel (like PNGs with transparency) would cause errors or incorrect predictions. If an issue occurs while loading the image, such as the file being missing, corrupted, or in an unsupported format, the *except* block will catch the exception, print an error message, and return early, preventing any further processing of an invalid image. This ensures that IDP can handle a variety of image inputs while gracefully managing any errors during file loading.

```
61 image = transform(image).unsqueeze(0).to(device)
```

Apply image transformations and prepare the image for inference. This line applies our selected transformations to the input image to match the expected format for the ResNet model. First *transform(image)* applies our image preprocessing pipeline for size, tensor, and pixel normalization. Then *.unsqueeze(0)* adds a batch dimension to the tensor. Since deep PyTorch expects images to be in this form, this step ensures that a single image can be processed correctly without triggering shape-related errors. Finally, *.to(device)* moves the processed image to the hardware.

```
63 #Perform Image Inference and Generate a Confidence Score
64 with torch.no_grad():
65     outputs = model(image)
66     probabilities = torch.nn.functional.softmax(outputs, dim=1)
67     confidence, predicted = torch.max(probabilities, 1)
68     damage_type = class_mapping[predicted.item()]
69
70 print(f"Predicted Damage Type: {damage_type} (Confidence: {confidence.item() * 100:.2f}%)")
```

Performing image classification inference. This section performs the inference using the trained model and creates a confidence score for the prediction. The process begins using the *with* blanket for *torch.no_grad* to disable any unnecessary gradient calculations to optimize performance and memory usage during inference. The input image, now preprocessed and correctly formatted, is passed through the model using *outputs = model(image)*, producing raw output classification scores. The line

probabilities = torch.nn.functional.softmax(outputs, dim=1) applies the softmax function to the model's classification scores, converting them into probability values that sum to 1 across all possible damage categories. Since the raw output (*outputs*) from the model consists of unnormalized scores, softmax is used to normalize them into interpretable probability distributions. The argument *dim=1* ensures that softmax is applied along the correct axis, meaning it processes each set of predictions separately when handling batch inputs. By transforming the classification scores into probabilities, the script is able to determine the likelihood of each damage category, making it possible to identify the prediction most likely to be correct. The next step extracts the most confident prediction by applying

torch.max(probabilities, 1), which identifies the highest probability value (confidence score) and the corresponding class index. The *confidence* variable stores the probability of the predicted class, and the *predicted* variable holds the associated numerical index. The index is converted into a human-readable damage category by using

damage_type = class_mapping[predicted.item()]. The *item()* method extracts the integer value from the tensor, ensuring it can be properly used as a key in the *class_mapping* dictionary. This step prevents any potential KeyError issues, ensuring the model's output aligns correctly with one of the predefined vehicle categories. The script prints the classification output using a formatted string that includes the predicted damage type and the confidence score converted into a percentage using *confidence.item() * 100*, formatted to two decimal places for readability. Displaying the confidence score alongside the predicted category improves interpretability, allowing users to assess how certain the model is about its classification. This structured

approach ensures that IDP provides both accurate and transparent predictions, offering users insight into the model's decision-making process.

```
72 # The Main Function and Input Sanitization
73 > if __name__ == "__main__":
74     while True:
75         image_path = input("Enter the path to the vehicle damage image (or type 'exit' to quit): ").strip()
76
77         if image_path.lower() == 'exit':
78             print("Exiting the system. Goodbye!")
79             break
80
81         # Remove surrounding quotes if present
82         image_path = image_path.strip('\"')
83
84         predict_damage(image_path)
```

The main function. The main function serves as the entry point for IDP, handling user input and input sanitization to ensure a smooth user experience. The script begins with the

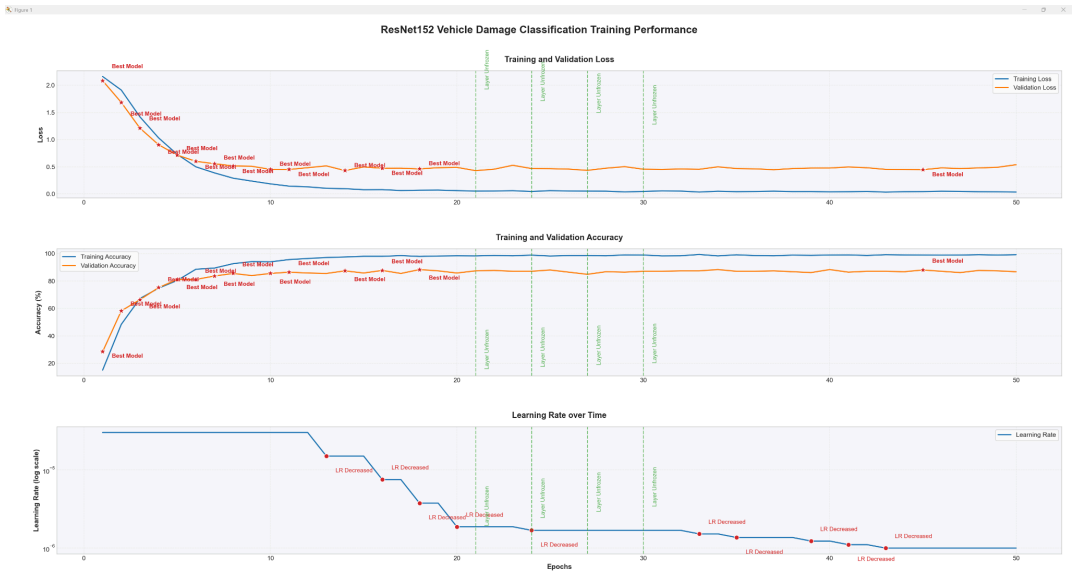
if __name__ == “__main__”:, which ensures that the program is self-executable rather than needing to be imported as a module in another program. This structure allows IDP to function independently while maintaining flexibility for integration into future projects if needed. Inside this block a ***while True:*** loop is used to continuously prompt the user for an image file path, allowing for multiple predictions in a single session. The script collects input using *input()*, displaying a message instructing the user to enter the file path of a vehicle damage image or to type *exit* to terminate the program. The *strip()* method is applied to remove any leading or trailing whitespace that may accidentally be included in the input. Immediately after, the script check sto see if the user typed exit, which is case insensitive due to using *.lower()*. If this

condition is met, a goodbye message is printed, and the ***break*** statement exits the loop and the program. To improve usability, the script sanitizes the input path input by calling ***image_path.strip(“”)*** . This step is particularly useful for Windows users who may copy and paste paths using the “copy as path” feature, which encloses the file path in quotes. Stripping these quotes ensures that the program correctly interprets the path without errors. Finally, the cleaned-up path input is passed to ***predict_damage(image_path)***, triggering the vehicle damage classification process. This structure allows the program to handle multiple user inputs while preventing common formatting issues making it flexible for real-world usage.

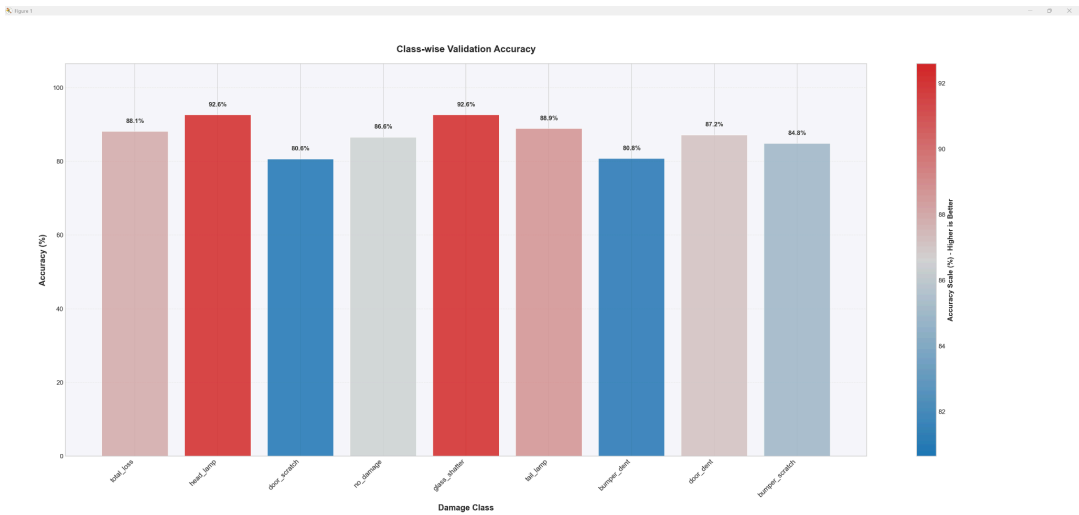
Results

In its highest-performing training cycle, VDRS achieved a validation accuracy of 90.91%, indicating that the model is particularly proficient in distinguishing between damage categories in the pre-labeled training dataset. To evaluate training dynamics, I asked Claude (2025) to implement a Matplotlib-based visualization system to track the training loss, validation accuracy, learning rate schedule, and progressive unfreezing. The LLM added about 200 lines of code to my existing training script, which I did not review, revise, or have any part in creating. As such, I will only show that program’s outputs and will not submit the program it created. Still, the results are useful and are worth exploring. Below are two visualization systems that provide a detailed breakdown of the model’s training progression, highlighting key points where significant events occurred throughout the 50 epochs. The first image is a graph depicting training validation and loss, and the second image is a bar chart of class-wise validation

accuracy, which highlights the strengths and weaknesses of the classifier across different categories.



(Claude, 2025)



(Claude, 2025)

Training Performance Line Graphs

The training performance graphs provide a comprehensive visualization of the learning process across an arbitrary training cycle of VDRS. The graph tracking training and validation loss demonstrates a clear downward trajectory, confirming that the model effectively minimizes classification errors over time. During early epochs, rapid loss reduction is observed, a characteristic behavior of transfer learning (Chilamkurthy, 2017) when fine-tuning a pre-trained model like ResNet152. The ‘best model’ markers indicate epochs where the validation accuracy or loss reached a new best epoch. By approximately epoch 10, the loss value starts to stabilize, showing diminishing gains in loss reduction, although validation loss fluctuates a bit beyond epoch 20, which is likely due to dataset limitations or model overfitting. The relatively small gap between training and validation loss suggests that the regularization techniques such as dropout and adaptive learning rate scheduling helped to prevent significant overfitting. The graph tracking training and validation accuracy reinforces these findings. The initial accuracy spike corresponds to the early-stage adjustments of the pre-trained model to the dataset. Again, around epoch 10, accuracy begins to stabilize, with training accuracy consistently outperforming validation accuracy. This is expected, as the model has been exposed to the training set multiple times, whereas the validation set remains unseen during weight updates. The lack of severe overfitting suggests that progressive unfreezing and batch normalization techniques successfully regularized the training process. The “layer unfrozen” markers indicate the moments when deeper layers were gradually unlocked for training, allowing the model to refine its understanding of vehicle damage patterns. These events correlate with fluctuations in validation accuracy, indicating that fine-tuning higher-level features led to periods of minor instability,

requiring careful adjustments in training parameters. The final graph tracking learning rate adjustments over time provides interesting insight into the adaptive optimization strategy. For this particular run, the adaptive learning rate *factor* was decreased to 10% (**0.9**) instead of the 50% (**0.5**) that ultimately proved most effective. The ‘LR Decreased’ markers represent epochs where the *ReduceLROnPlateau* scheduler dynamically lowered the learning rate in response to stagnation in validation performance. The stepwise reductions around epochs 10, 20, and 30 suggest that as the model approached convergence, it required smaller, more precise weight updates to continue improving classification accuracy. This gradual reduction in learning rate prevented sudden shifts in the model’s learned representations and helped to achieve stable convergence. Overall, these training performance graphs show an effective model.

Class-Wise Validation Accuracy Bar Graph

The bar graph represents a breakdown of the validation accuracy for each damage category, offering insight into how well the model differentiates between various types of damage. Each bar represents a specific damage class, with the height corresponding to validation accuracy. The color gradient scale provides visual reinforcement of the accuracy distribution, with red tones representing the ‘hot’, highest performing categories, and the blue tones representing the ‘cold’, lower performing ones. The highest performing classes were ‘headlamp’ and ‘glass-shatter’, both achieving accuracy greater than 92%. This suggests that the model is good at detecting large, distinct visual features such as broken glass and damaged headlamps. These types of damage likely have high contrast, well-defined edges, and strong feature

extraction cues, making them easier to classify. Similarly, ‘total loss’ and ‘tail lamp’ also scored well, further confirming that the model is particularly effective at identifying damage with these visual features. The lowest performing classes were ‘door dent’ and ‘door scratch’, with each scoring just over 80% (which is still not bad.) This indicates that the model struggled with subtle, surface-level damage. This performance gap could be due to class imbalance in the dataset, poor feature differentiation between scratches and dents under varying lighting conditions, or similar visual feature overlap between the different classes. The fact that ‘no damage’ scored relatively well suggests that the model does have a solid capability to distinguish between damaged and undamaged vehicles, but minor imperfections are likely to introduce statistical noise.

Conclusion

Developing the VDRS and IDP systems has been an immensely rewarding experience for me, marking the culmination of many years of study, personal interest, and practical experience in software development. This project has been a long-term goal of mine throughout my higher education, and witnessing it come together has been incredibly satisfying. In terms of success, I would characterize the project as generally successful, with room for growth. All of the intended features are functioning as planned, and the model has shown strong validation accuracy in distinguishing between the major damage categories. However, there is always room for improvement. One of the most significant challenges was having access to limited training data, which needed substantial augmentation through artificial techniques. If I were working for a company with access to professional-quality data, I am confident that I could create a significantly more versatile and reliable model, given my current level of education. This experience has boosted my confidence in pursuing a career in the AI/ML specialization of the

insurance industry. I now have a solid foundation of competency in building, fine-tuning, and deploying AI systems, and I have demonstrated the ability to overcome challenges using effective AI techniques. I feel more prepared to compete in the job market for roles related to deep learning and AI-driven software development. I am looking forward to the next class in the degree program that relates to computer vision. Having successfully designed and implemented a relatively sophisticated convolutional neural network for damage identification, I am confident that I can contribute meaningfully to an industry team and continue pushing the boundaries of what AI systems can achieve.

This project not only tested my technical skills but also expanded my understanding of deep learning, model optimization, and AI implementation. This accomplishment was made easier by the PyTorch toolkits, as well as the deliberate use of large language models (LLMs) like Grimoire (2025), Claude Sonnet 3.7 (2025), and OpenAI's Deep Research (2025.) LLMs were not a shortcut, nor were they a replacement for my own understanding, but rather an augmentation of my problem-solving ability. Through careful review, testing, and validation of every LLM output, I made sure that I fully understood what the code was doing before incorporating it into the project, and there is nothing in VDRS that is beyond my comprehension as a developer. Using LLMs as a collaborative partner rather than a shortcut to get my homework done quicker allowed me to accelerate my understanding of how these systems work together, and attempt a much more sophisticated program than I would have been comfortable creating without the help of these tools. This experience has been a defining moment in my learning journey, showing that long-term dedication, structured planning, and access to powerful tools can lead to real and impactful results. Having success at designing this CNN-based system has been

an important milestone, but I plan on continuing to improve and refine it as I gain more experience, ensuring it evolves alongside my growing mastery of AI and ML systems.

References

- Analytics Vidhya. (2024, December 17). Imbalanced data: How to handle imbalanced classification problems. Analytics Vidhya. Retrieved March 9, 2025, from: <https://www.analyticsvidhya.com/blog/2017/03/imbalanced-data-classification/>
- Bhattacharjee, R., Rittler, N., & Chaudhuri, K. (2024). Beyond discrepancy: A closer look at the theory of distribution shift. arXiv. Retrieved March 9, 2025, from: <https://arxiv.org/pdf/2405.19156>
- Chilamkurthy, S. (2017). Transfer learning for computer vision tutorial. PyTorch Tutorials. Retrieved January 26, 2025, from: https://pytorch.org/tutorials/beginner/transfer_learning_tutorial.html
- Clark, J. A., & Contributors. (n.d.). Pillow 11.1.0 documentation. Read the Docs. Retrieved March 9, 2025, from: <https://pillow.readthedocs.io/en/stable/>
- Claude 3.7 Sonnet. (2025, March 9). Enhancing vehicle damage classification system [Personal communication]. Retrieved March 7, 2025, from: <https://claude.ai/>
- DataCamp. (2024, October 15). Normalization vs. standardization: Key differences explained. DataCamp. Retrieved March 9, 2025, from: <https://www.datacamp.com/tutorial/normalization-vs-standardization>

Deep Research. (2025). Deep Research: Advanced AI-Assisted Model Development. OpenAI.

Retrieved January 25 through March 7, 2025, from: <https://openai.com/deep-research>

Dhakal, S. (2024, September 28). Understanding normalization in deep learning: Why it's crucial

for training neural networks. Sisir's Blog. Retrieved March 9, 2025, from:

<https://sisirdhakal.hashnode.dev/understanding-normalization-in-deep-learning-why-its-crucial-for-training-neural-networks>

Fairlie, M. (2025, January 6). Decision support systems applications and uses. Business.com.

Retrieved March 9, 2025, from:

<https://www.business.com/articles/decision-support-systems-dss-applications-and-uses/>

GeeksforGeeks. (2025, January 27). Residual networks (ResNet) in deep learning.

GeeksforGeeks. Retrieved February 2, 2025, from:

<https://www.geeksforgeeks.org/residual-networks-resnet-deep-learning/>

Google Developers. (n.d.). Overfitting | Machine Learning Crash Course. Google Developers.

Retrieved February 2, 2025, from:

<https://developers.google.com/machine-learning/crash-course/overfitting/transforming-data>

Gavrikov, P. (2022, October 28). Downloading and using the ImageNet dataset with PyTorch.

Towards Data Science. Retrieved March 9, 2025, from:

<https://medium.com/towards-data-science/downloading-and-using-the-imagenet-dataset-with-pytorch-f0908437c4be>

Grimoire. (2025). Conversation about PyTorch Damage Classification Projects. OpenAI.

Retrieved January 25 through March 9, 2025, from: <https://chat.openai.com>

Hugging Face. (n.d.). EfficientNet model documentation. Hugging Face. Retrieved February 2,

2025, from: https://huggingface.co/docs/transformers/en/model_doc/efficientnet

Kaggle. (n.d.). Public API. Kaggle. Retrieved March 9, 2025, from:

<https://www.kaggle.com/docs/api>

Kingma, D. P., & Ba, J. (2017). Adam: A method for stochastic optimization. arXiv. Retrieved

March 9, 2025, from: <https://arxiv.org/pdf/1412.6980>

laridzhang. (2019, September 29). Why Pytorch officially use mean=[0.485, 0.456, 0.406] and

std=[0.229, 0.224, 0.225] to normalize images? Stack Overflow. Retrieved March 9,

2025, from:

<https://stackoverflow.com/questions/58151507/why-pytorch-officially-use-mean-0-485-0-456-0-406-and-std-0-229-0-224-0-2>

Latope2-150. (2018, March 27). Should I remove Dropout layer when testing my trained model?

PyTorch Forums. Retrieved March 9, 2025, from:

<https://discuss.pytorch.org/t/should-i-remove-dropout-layer-when-testing-my-trained-model/15581>

Manssor, H. (n.d.). Car damage assessment [Data set]. Kaggle. Retrieved January 25, 2025,

from: <https://www.kaggle.com/datasets/hamzamanssor/car-damage-assessment>

Padmanabhan, R., Thunga Padmanabhan, D. R., & Thanigaivelu, K. (2023). Convolutional

neural networks for vehicle damage detection. AIP Conference Proceedings, 2790(1),

020095. Retrieved February 2, 2025, from: <https://doi.org/10.1063/5.0152899>

Pandas Development Team. (n.d.). Pandas Documentation. Pandas. Retrieved March 9, 2025,

from: <https://pandas.pydata.org/docs/>

PyTorch. (2021). Datasets & DataLoaders. PyTorch Tutorials. Retrieved March 9,

2025, from: https://pytorch.org/tutorials/beginner/basics/data_tutorial.html

PyTorch. (2022). Quickstart. PyTorch Tutorials. Retrieved January 25, 2025, from:

https://pytorch.org/tutorials/beginner/basics/quickstart_tutorial.html

PyTorch. (2024a). Sequential. In PyTorch documentation. Retrieved March 9, 2025 from:

<https://pytorch.org/docs/stable/generated/torch.nn.Sequential.html>

PyTorch. (2024b). torch.optim.lr_scheduler.ReduceLROnPlateau. PyTorch Documentation.

Retrieved March 9, 2025, from:

https://pytorch.org/docs/stable/generated/torch.optim.lr_scheduler.ReduceLROnPlateau.html

PyTorch. (2024c). Training a classifier. PyTorch Tutorials. Retrieved January 25,

2025, from: https://pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html

Regex Tutorial. (n.d.). Regular expression metacharacters. Regex Tutorial. Retrieved March 9,

2025, from: <https://www.regextutorial.org/regular-expression-metacharacters.php>

SciKit-Learn Developers. (n.d.). sklearn.model_selection.train_test_split. In scikit-learn 0.20.4

documentation. Retrieved March 9, 2025, from:

https://scikit-learn.org/0.20/modules/generated/sklearn.model_selection.train_test_split.html

Sharma, P. (2023, June 20). How to train an image classification model in PyTorch and

TensorFlow? Analytics Vidhya. Retrieved January 26, 2025, from:

<https://www.analyticsvidhya.com/blog/2020/07/how-to-train-an-image-classification-model-in-pytorch-and-tensorflow/>

Sling Academy. (2024, December 14). A step-by-step tutorial on fine-tuning classification

models in PyTorch. Sling Academy. Retrieved January 26, 2025, from:

<https://www.slingacademy.com/article/a-step-by-step-tutorial-on-fine-tuning-classification-models-in-pytorch/>

Stack Exchange Community. (2020, May 12). Artificially expanding the datasets through

rotation of images in MNIST. Data Science Stack Exchange. Retrieved February 2, 2025,

from:

<https://datascience.stackexchange.com/questions/71950/artificially-expanding-the-datasets-through-rotation-of-images-in-mnist>

Stewart, D. P. (2023, June 20). Detecting vehicle damage with advanced deep learning

technology. Medium. Retrieved February 2, 2025, from:

<https://medium.com/@dallinpstewart/detecting-vehicle-damage-with-advanced-deep-learning-technology-52a26a7ca796>

TorchVision Developers. (n.d.). TorchVision stable documentation. PyTorch. Retrieved March 7,

2025, from: <https://pytorch.org/vision/stable/index.html>